

Learn C The Hard Way - 1

남궁명수

이 책은 사실 C에 관한 책이 아니다
속았다고 느끼지는 말아 달라. 하지만 C 프로그래밍을 가르치기 위한 책이 아니다.
물론 C로 프로그램을 작성하는 법을 배우게 될 것이다. 그러나 이 책에서 가장 중요한
교훈은 철저한 방어적 프로그래밍이다.

오늘날 너무 많은 프로그래머들이 자신이 작성한 코드가 잘 동작할 것이라고 그냥 가정한다.
하지만 언젠가는 그것이 치명적으로 실패하게 된다. 이는 특히 많은 문제를 대신 해결해주는
현대적인 언어들만 배워온 사람들에게서 더욱 두드러진다. 이 책을 읽고 내가 제시하는
연습을 따라가다 보면, 악의적인 공격과 결함으로부터 스스로를 방어하는 소프트웨어를
만드는 방법을 배우게 될 것이다.

내가 C를 사용하는 이유는 아주 분명하다 C는 망가진 언어이다.
1970년대에는 타당했지만, 지금은 전혀 말이 되지 않는 설계 결정들로 가득 차 있다. 제한
없는 포인터 사용부터 심각하게 잘못 설계된 NULL 종료 문자열까지, C에서 발생하는 거의
모든 보안 결함의 원인이 여기에 있다.

내 생각에 C는 너무나도 망가진 언어라서, 아직 널리 사용되고 있음에도 불구하고 안전하게
작성하기 가장 어려운 언어다. 솔직히 말해, 나는 어셈블리어가 오히려 C보다 안전하게
작성하기 쉽다고 생각한다. 더 솔직히 말하자면, 누구도 새 C 코드를 작성해서는 안된다고
생각한다.

그렇다면 왜 내가 C를 가르치느냐고?
그 이유는 당신을 더 강하고 더 나은 프로그래머로 만들고 싶기 때문이다. 그리고 C는 그
목적에 아주 훌륭한 언어다. 이유는 두 가지다.

첫째, C에는 거의 모든 현대적인 안정장치가 없다. 따라서 당신은 더 경계해야 하고, 더
많은 것을 인식해야 한다. C로 안전하고 탄탄한 코드를 작성할 수 있다면, 어떤 언어로든
그런 코드를 작성할 수 있다. 여기서 배우는 기술은 앞으로 사용되는 모든 언어에 그대로
적용된다.

둘째, C를 배우면 방대한 레거시 코드에 직접 접근할 수 있고, 수많은 파생 언어들의 기본
문법을 이해할 수 있다. C를 배우고 나면 C++, Java, Objective-C, JavaScript는 물론, 다른
언어들까지 훨씬 쉽게 배울 수 있다.

이런 이야기를 해서 당신을 겁주고 싶지는 않다. 왜냐하면 나는 이 책을 굉장히 재미있고,
쉽고, 교묘하게 만들 계획이기 때문이다.

다른 언어에서는 해보지 않았을 프로젝트들로 C 학습을 재미있게 만들 것이고, 내가
검증해온 연습 패턴을 사용해 천천히 실력을 쌓게 할 것이다 그리고 일부러 코드를 깨뜨린
다음 고치게 만들어서, 왜 이런 문제들이 중요한지를 직접 이해하게 할 것이다.

스택 오버플로우, 불법 메모리 접근, 그리고 C 프로그램을 괴롭히는 다른 일반적인 결함들을 직접 만들어보게 될 것이다. 그래야 무엇과 싸우고 있는지 알 수 있다.

이 책을 끝까지 마치는 일은 쉽지 않을 것이다. 하지만 무사히 다 마쳤을 때, 당신은 훨씬 더 자신감 있고 유능한 프로그래머가 되어 있을 것이다.

정의되지 않은 동작 신봉자들

이 책을 끝낼 즈음이면, 당신은 마주치는 거의 모든 C 프로그램을 디버그하고, 읽고, 고치고, 필요하다면 새롭고 탄탄한 C 코드까지 작성할 수 있을 것이다. 하지만 나는 공식적인 C만을 가르치지 않는다.

C 언어 자체를 배우고 잘 사용하는 법을 배우겠지만, 공식 C는 그다지 안전하지 않다. 대부분의 C 프로그래머들이 탄탄한 코드를 작성하지 못하는 이유는 정의되지 않은 동작(Undefined Behavior, UB) 때문이다.

UB는 ANSI C 표준의 일부로, 컴파일러가 당신이 작성한 코드를 무시해도 되는 모든 경우를 나열해 놓은 것이다. 실제로 표준에는 “이런 코드를 작성하면, 이후의 동작에 대해 아무 보장도 하지 않는다”라고 명시된 부분이 있다.

UB는 C 프로그램이 문자열의 끝을 넘어 읽을 때 발생한다. 이는 C에서 매우 흔한 오류다. C에서 문자열은 NULL 바이트(0)로 끝나는 메모리 블록으로 정의된다. 그런데 많은 문자열은 프로그램 외부에서 들어오며, 종종 이 NULL 바이트가 없다. 그러면 C 프로그램은 문자열의 끝을 넘어 컴퓨터의 메모리를 읽게 되고, 결국 프로그램이 충돌한다.

C 이후에 만들어진 대부분의 언어들은 이러한 문제를 방지하려고 하지만, C는 아무것도 하지 않는다. C는 UB를 막기 위해 거의 아무 일도 하지 않기 때문에, 많은 C 프로그래머들은 “그건 UB니까 내가 신경 쓸 필요 없다”라고 생각한다. 이들은 NULL 바이트 초과 읽기를 포함한 코드들을 작성해 놓고, 문제를 지적하면 이렇게 말한다.

“그건 UB야, 내가 막을 필요 없어”

이렇게 UB에 의존하는 태도 때문에, 대부분의 C 코드는 끔찍할 정도로 안전하지 않다.

나는 UB를 피하기 위해, UB를 발생시키지 않는 코드를 작성하거나, 발생하지 않도록 방어하는 코드를 작성한다. 하지만 UB가 너무 많아서 이것은 서로 얽힌 함정들로 가득한 고르디우스의 매듭과도 같다.

이 책을 진행하면서, 나는 UB를 발생시키는 방법, 가능하다면 피하는 방법, 그리고 다른 사람의 코드에서 UB를 발생시키는 방법까지 보여줄 것이다. 하지만 솔직히 말해, UB의 거의 무작위적인 특성을 완전히 피하는 것은 거의 불가능하다. 최선을 다할 수밖에 없다.

경고

강성 C 팬들은 종종 UB 를 가지고 당신을 공격하려 들 것이다.

이들은 실제 C 코드를 거의 작성하지 않으면서, UB 만 외워서 초보자를 지적으로 공격하는 사람들이다. 이런 공격적인 프로그래머를 만나면 무시하라. 이들은 도움을 주기보다는 자신의 우월함을 증명하려 끝없는 질문만 던질 뿐이다.

C 는 아름답고 추한 언어다.

UB 의 존재는 C 를 배우는 것이 좋은 또 하나의 이유다. 내가 가르치는 방식대로 좋은 C 코드를 작성할 수 있다면, 어떤 언어에서도 살아남을 수 있다.

긍정적인 면에서 C 는 매우 우아한 언어다. 문법은 놀라울 정도로 작지만 강력하다. 지난 45 년 동안 수많은 언어들이 C 의 문법을 차용한 이유가 있다. 적은 기술로 많은 것을 할 수 있다.

C 를 배우고 나면, 아름답지만 동시에 조금은 추한 것에 대한 감각을 갖게 될 것이다. 오래된 기념비처럼, 멀리서 보면 멋지지만 가까이 다가가면 균열과 결함이 보인다.

그래서 나는 최신 컴파일러에서 작동하는 가장 최근의 실용적인 C 를 가르칠 것이다. 단순하고, 직관적이며, 어디서든 작동하고, 많은 함정을 피하는 C 다. 이것이 내가 실제 작업에 사용하는 C 다. 강성 팬들이 집착하는 백과사전식 C 가 아니다.

나는 20 년 동안 대규모 시스템을 안정적으로 운영한 C 코드를 작성해왔기 때문에, 이 방식이 탄탄하다는 것을 안다. 내 코드는 Twitter, Aribnb 같은 회사의 시스템에서 수조 건의 트랜잭션을 철했다. 거의 실패하지 않았고, 보안 공격도 막아냈다.

중요한 것은 코드 스타일이 아니라 마인드셋이다. 나는 항상 “아무것도 제대로 작동하지 않을 것”가정하고 코드를 작성한다. 많은 프로그래머들은 UB 나 운영체제가 자신을 구해줄 것이라고 믿지만, 그것은 해결책이 아니다.

만약 누군가 이 책의 코드가 “진짜 C 가 아니다”라고 말한다면, 그 사람이 나와 같은 실적을 가지고 있는지 생각해보라.

내 코드가 완벽하냐고? 아니다. 완벽한 C 코드는 존재하지 않는다. 어떤 언어에서도 마찬가지다. 모든 코드는 결함이 있다. 차이는, 나는 항상 내 코드가 결함이 있다고 가정하고 그것을 막으려고 한다는 점이다.

이 책을 끝까지 마치면, 내가 지난 20 년간 사용해온 방어적 프로그래밍 사고 방식을 얻게 될 것이다.

이 책에서 배우게 될 것

이 책의 목적은 당신이 C로 직접 소프트웨어를 작성하거나, 다른 사람의 C 코드를 수정할 수 있을 만큼 강해지게 만드는 것이다. 이후에는 K&R C(The C Programming Language 2 판)를 읽기를 권한다.

이 책에서 가르치는 것은 다음과 같다.

- C 문법과 관용구의 기초
- 컴파일, make 파일, 링커
- 방어적 코딩 기법
- C 코드 깨뜨리기
- 기본적인 UNIX 시스템 소프트웨어 작성

이 책을 읽는 방법

이 책은 이미 다른 언어를 하나 잇아 배운 사람을 위한 책이다.

프로그래밍이 처음이라면 Learn Python the Hard Way 를 먼저 읽어라.

이 책은 일반적인 설명 위주의 책이 아니다. 각 연습마다 영상 강의가 있고, 바로 코드를 작성한 뒤 설명을 듣는다. 추상적인 설명보다 훨씬 효과적이기 때문이다.

반드시 지켜야 할 규칙

- 강의를 먼저 본다.
- 코드를 직접 타이핑 한다.
- 주석까지 그대로 입력한다.
- 같은 출력이 나오는지 확인한다.
- 버그를 고친다.
- Extra Credit 에 도전한다.
- 먼저 스스로 해결하려고 노력한다.

핵심 역량

아마 당신은 Python, Ruby 같은 “느슨한” 언어를 써왔을 것이다. 혹은 LISP, Prolog, IDE 에 의존하는 환경일지도 모른다. 상관없다. 대부분의 사람들은 다음이 부족하다.

읽기와 쓰기

세부사항에 대한 주의

차이점 식별

계획과 디버깅

C는 당시에게 이것들을 강제로 훈련시킨다. 대충 써서는 아무것도 작동하지 않는다. 그게 C의 장점이다. 단순하지만, 스스로 파악할 수 있고, 기계와 프로그래밍의 본질을 이해하게 만든다.

Exercise 0. 환경 설정.....	10
[0-1] Linux.....	10
[0-2] Mac OS X.....	11
[0-3] Windows	11
[0-4] 텍스트 편집기	11
[0-5] IDE 를 사용하지 마시오	12
Exercise 1. 컴파일러 먼지 털기(Dust Off That Compiler).....	13
[1-1] ex1.c.....	13
[1-1-1] 코드 분석(Breaking It Down).....	13
[1-1-2] 줄별 설명.....	13
[1-1-3] 실행 결과.....	15
[1-2] 망가뜨리는 방법(How to Break It)	15
[1-3] 추가 과제(Extra Credit)	16
Exercise 2. Makefiles 를 사용한 빌드	17
[2-1] Make 사용하기	17
[2-2] ex2.1.mak.....	18
[2-3] 실행 결과	19
[2-4] 일부러 망가뜨려 보기	19
[2-4-1] Make & Makefile 핵심 요약.....	20
[2-5] 추가 과제(Extra Credit)	21
Exercise 3. 서식화된 출력	22
[3-1] ex3.c.....	22
[3-1] 결과로 보여야 할 것(What You Shoud See).....	23
[3-2] 외부 조사(External Research).....	23
[3-3] 망가뜨려 보기(How to Break It)	24
[3-4] 추가 과제(Extra Credit)	25
Exercise 4. Using a Debugger (GDB, LLDB).....	28
[4-1] GDB 기초 설명.....	28
[4-2] GDB 사용 설명.....	28
[4-3] LLDB 기초 설명.....	29
[4-4] LLDB 기본 명령어	31
Exercise 5. C 연산자 암기하기	32
[5-1] 왜 암기가 필요한가	32
[5-2] 암기에 대한 오해에 대한 경구.....	32
[5-3] C 문법을 암기하는 구체적인 방법.....	32

[5-4] 암기는 양방향으로 해야 한다.....	33
[5-5] 연산자 목록(*플래시카드).....	33
Exercise 6. C 문법 암기하기.....	36
[6-1] The Keywords(*플래시카드)	36
[6-2] Syntax Structures(문법 구조)	37
[6 -2-1] if 문	38
[6 -2-2] switch 문	38
[6 -2-3] while 반복문.....	38
[6 -2-4] do-while 반복문.....	39
[6 -2-5] for 반복문	39
[6 -2-6] enum	39
[6 -2-7] goto.....	39
[6 -2-8] 함수 정의	40
[6 -2-9] typedef.....	40
[6 -2-10] struct.....	40
[6 -2-11] union.....	41
[6-3] 격려의 말	41
[6-4] 주의의 말	41
Exercise 7. 변수와 타입.....	43
[7-1] ex7.c	43
[7-2] 실행 결과	44
[7-3] 망가뜨려 보기(How to Break It)	45
[7-4] 추가 과제(Extra Credit)	45
Exercise 8. if, else-if, else.....	46
[8-1] ex8.c	46
[8-2] 실행 결과 예시	47
[8-3] 망가뜨려 보기(How to Break It)	48
[8-4] 추가 과제(Extra Credit).....	48
Exercise 9. while 루프와 Boolean 표현식	49
[9-1] ex9.c	49
[9-2] 실행 결과 예시	49
[9-3] 망가뜨려 보기(How to Break It)	50
[9-4] 추가 과제(Extra Credit)	50
Exercise 10. Switch 문.....	51
[10-1] ex10.c	51

[10-2] swtich 문의 동작 원리.....	52
[10-3] 실무에서 기억할 점	52
[10-4] 실행 결과 예시.....	52
[10-5] 망가뜨려 보기(How to Break It)	53
[10-6] 추가 과제(Extra Credit).....	53
Exercise 10. Switch 문	54
[10-1] ex10.c	54
[10-2] swtich 문의 동작 원리.....	55
[10-3] 실무에서 기억할 점	55
[10-4] 실행 결과 예시.....	55
[10-5] 망가뜨려 보기(How to Break It)	56
[10-6] 추가 과제(Extra Credit).....	56
Exercise 11. 배열과 문자열	57
[11-1] ex11.c	57
[11-2] 핵심 포인트	58
[11-3] 실행 결과 예시.....	59
[11-4] 망가뜨려 보기(How to Break It)	59
[11-5] 추가 과제(Extra Credit).....	59
Exercise 12. 배열과 크기	60
[12-1] ex12.c	60
[12-2] 핵심 포인트	61
[12-3] 실행 결과 예시.....	61
[12-4] 망가뜨려 보기(How to Break It)	61
[12-5] 추가 과제(Extra Credit).....	62
Exercise 13. for 문과 문자열	63
[13-1] ex13.c	63
[13-2] for 문 구조.....	64
[13-3] 명령줄 인수 처리	64
[13-4] 실행 결과 예시.....	64
[13-5] 문자열 배열 이해	65
[13-6] 망가뜨려 보기(How to Break It)	65
[13-7] 추가 과제(Extra Credit).....	65
Exercise 14. 함수 작성과 사용	66
[14-1] ex14.c	66
[14-2] 설명	67

[14-3] 실행 결과 예시.....	68
[14-4] 망가뜨려 보기(How to Break It).....	68
[14-5] 추가 과제(Extra Credit).....	69
Exercise 15. 포인터, 두려운 포인터	70
[15-1] ex15.c	70
[15-2] 실행 결과 예시.....	71
[15-3] 포인터 설명	72
[15-4] 포인터는 배열이 아니다	73
[15-5] 망가뜨려 보기(How to Break It).....	73
[15-6] 추가 과제(Extra Credit).....	73
Exercise 16. 구조체와 그에 대한 포인터.....	74
[16-1] ex16.c	74
[16-2] 프로그램 설명 방식	76
[16-3] 찾아본 것.....	77
[16-4] 실행 결과 예시.....	78
[16-5] 구조체 설명	79
[16-6] 망가뜨려 보기(How to Break It).....	79
[16-7] 추가 과제(Extra Credit).....	79
Exercise 17. 힙과 스택 메모리 할당	80
[17-1] ex17.c	80
[17-2] 프로그램 개요.....	85
[17-3] 실행 결과 예시.....	87
[17-4] 힙과 스택 메모리 할당.....	88
[17-5] 스택과 힙에서 발생할 수 있는 문제	88
[17-6] Database_open / Database_close 를 만들었는가	88
[17-7] 망가뜨려 보기(How to Break It).....	89
[17-8] 추가 과제(Extra Credit).....	89
[17-9] 찾아본 것.....	89
[17-10] 파일 기반 구조체 데이터베이스 프로그램 분석 및 설계 사고 과정	92
Exercise 18. 함수 포인터	93
[18-1] ex18.c	93
[18-2] 함수 호인터 형식	96
[18-3] 함수 포인터를 매개변수로 전달하는 문제.....	97
[18-4] 실행 결과 예시.....	98
[18-5] 망가뜨려 보기(How to Break It).....	99

[18-6] 추가 과제(Extra Credit).....	99
Exercise 19. Zed 의 끝내주는 디버그 매크로.....	100
[19-1] C 의 에러 처리 문제.....	100
[19-2] 디버그 매크로.....	101
[19-3] dbg.h 코드.....	101
[19-4] dbg.h 의 모든 매크로를 실제 실행해보는 테스트 코드.....	105
[19-5] 왜 die 함수 대신 매크로를 사용하는지.....	108
[19-6] 추가 과제(Extra Credit).....	108
Exercise 19-1. Zed 의 끝내주는 디버그 매크로 상세 설명	109
Exercise 20. 고급 디버깅 방법	115
[20-1] 디버그 출력 vs GDB.....	115
[20-2] 언제 무엇을 쓰는가	116
[20-3] 디버깅 전략	117
[20-4] 추가 과제(Extra Credit).....	117
Exercise 20-1. 연습하기	118
[20-1] macOS 환경.....	118
[20-2] Linux 환경	122
[20-3] C 프로그램 크래시 원인 분석 및 LLDB 디버깅 실습	124

Exercise 0. 환경 설정

전통적인 첫 번째 연습문제인 Exercise 0 은, 이 책의 나머지 내용을 진행하기 위해 컴퓨터 환경을 설정하는 단계이다. 이 연습에서는 여러분이 사용하는 컴퓨터 종류에 따라 필요한 패키지와 소프트웨어를 설치하게 된다.

이 연습을 따라 하다가 문제가 생기면, 자신의 컴퓨터에 해당하는 Exercise 0 영상을 시청하고 그 설정 과정을 그대로 따라하자. 그 영상에서는 각 단계를 어떻게 수행하는지 보여 주며, 발생할 수 있는 문제들을 해결하는 데 도움을 준다.

[0-1] Linux

Linux 는 C 개발 환경을 설정하기에 가장 쉬운 운영체제이다.

Debian 계열 시스템에서는 터미널에서 다음 명령을 실행하시오.

```
$ sudo apt-get install build-essential
```

Fedora, RedHat, CentOS 7 과같은 RPM 기반 Linux 에서는 다음과 같이 설치한다.

```
$ sudo yum groupinstall development-tools
```

다른 Linux 배포판을 사용 중이라면, “c development tools”와 사용 중인 Linux 이름을 함께 검색하여 필요한 패키지를 찾으면 된다.

설치가 끝나면 다음 명령을 입력해보자.

```
$ cc --version
```

설치된 컴파일러 정보를 확인할 수 있을 것이다. 대부분의 GNU C 컴파일러(GCC)가 설치되어 있을 것이지만, 책에서 사용하는 것과 다른 컴파일러여도 걱정할 필요는 없다.

원한다면 Clang C 컴파일러를 설치할 수도 있다. Linux 용 Clang 시작 가이드를 참고하거나, 문제가 생기면 온라인에서 검색해 보자.

[0-2] Mac OS X

Mac OS X에서는 설치가 더 쉽다.

먼저 Apple에서 최신 Xcode를 다운로드하거나, 설치 DVD가 있다면 DVD로 설치하자. 다운로드 파일은 매우 크기 때문에 시간이 오래 걸릴 수 있어 DVD 설치를 권장한다.

또한 “installing xcode”로 검색하면 설치 방법을 쉽게 찾을 수 있다. App Store를 통해 일반 앱처럼 설치할 수도 있으며, 이 경우 자동으로 업데이트를 받을 수 있다.

C 컴파일러가 제대로 설치되었는지 확인하려면 다음 명령을 입력하시오.

```
$ cc --version
```

Clang C 컴파일러 버전이 표시될 것이다. 만약 Xcode가 오래된 버전이라면 GCC가 설치되어 있을 수도 있는데, 어느 쪽이든 문제 없다.

[0-3] Windows

Microsoft Windows에서는 Cygwin을 사용해 표준 UNIX 개발 도구들을 설치하는 것을 추천한다. 설치와 사용은 비교적 쉽지만, 이 Exercise의 영상을 보면서 따라 하는 것이 가장 좋다.

Cygwin의 대안으로 MinGW를 사용할 수도 있다. MinGW는 더 단순한 시스템이지만 역시 사용할 수 있다.

다만, Microsoft 자사 개발 도구에서 C 지원을 점점 줄이고 있는 추세이므로, Microsoft 컴파일러로는 이 책의 코드를 빌드하는 데 문제가 생길 수 있다.

조금 더 고급 방법으로는 VirtualBox를 사용해 Linux 배포판을 설치하고, Windows 위에서 완전한 Linux 시스템을 실행하는 것이다. 이 방법의 장점은 가상 머신을 완전히 망가뜨려도 Windows 설정에는 전혀 영향을 주지 않는다는 점이다.

또한 Linux를 배우는 좋은 기회이기도 하며, 이는 재미있고 프로그래머로서 성장하는 데 매우 도움이 된다. 현재 Linux는 많은 분산 시스템과 클라우드 인프라 기업에서 주요 운영체제로 사용되고 있다. Linux를 배우는 것은 미래 컴퓨팅에 대해 이해를 크게 향상시켜 줄 것이다.

[0-4] 텍스트 편집기

프로그래머에게 텍스트 편집기를 선택하는 일은 쉽지 않다.

초보자라면 Gedit을 사용하는 것을 추천한다. 단순하고 코드 작성에 잘 맞기 때문이다. 다만, 특정 국제 환경에서는 제대로 동작하지 않을 수 있으며, 이미 프로그래밍 경험이 있다면 자신이 선호하는 편집기가 있을 수도 있다.

그래서 여러분에게 권하고 싶은 것은, 플랫폼에 맞는 몇 가지 대표적인 프로그래머용 텍스트 편집기를 직접 써보고 가장 마음에 드는 것을 하나 고르는 것이다.

이미 Gedit 을 쓰고 있고 만족하다면 그대로 사용하자. 다른 것을 써보고 싶다면, 모든 텍스트 편집기는 각자 이상한 단점이 있다. 하나를 골라 쓰다가, 나중에 더 마음에 드는 것을 발견하면 바꾸면 된다. 며칠씩 설정에만 매달리지 말자.

다음은 시도해 볼 수 있는 텍스트 편집기

- Linux 와 OS X 의 GEdit
- OS X 의 TextWrangler
- 터미널에서 실행되며 거의 모든 환경에서 동작하는 Nano
- Emacs 및 OS X 용 Emacs(학습 필요)
- Vim 및 MacVim

사람마다 맞는 편집기는 다르지만, 위의 편집기들은 무료이면서 잘 동작하는 것들이다. 필요하다면 유료 편집기도 포함해 여러 개를 시험해 보고 마음에 드는 것을 선택하자.

[0-5] IDE 를 사용하지 마시오

언어를 배우는 동안에는 통합 개발 환경(IDE) 사용을 피하라. IDE 는 작업을 빠르게 끝내는 데는 도움이 되지만, 언어를 깊이 이해하는 데는 오히려 방해가 되는 경우가 많다.

제 경험상, 실력이 뛰어난 프로그래머들은 IDE 를 사용하지 않으며, IDE 사용자와 비슷한 속도로 코드를 작성한다. IDE 로 작성된 코드는 품질이 더 낮은 경우가 많다. 왜 그런지는 정확히 모르겠지만, 언어에 대한 깊고 탄탄한 실력을 원한다면 학습 단계에서는 IDE 를 피하는 것을 강력히 추천한다.

전문 프로그래머용 텍스트 편집기를 다룰 줄 아는 능력은 직업적으로도 매우 유용하다. IDE 에 의존하면, 새로운 언어를 배우기 위해 IDE 가 그 언어를 지원할 때까지 기다려야 하는 상황이 생긴다.

이는 커리어에 비용을 발생시킨다. 언어의 유행 변화에 뒤처지게 만들기 때문이다. 반면, 일반적인 텍스트 편집기를 사용하면 언제든지 어떤 언어든 바로 코딩할 수 있다. 누구의 허락도 필요 없다.

범용 텍스트 편집기는 자유를 의미한다.

스스로 탐구하고, 원하는 방향으로 커리어를 관리할 수 있는 자유이다.

Exercise 1. 컴파일러 먼지 털기(Dust Off That Compiler)

모든 설치가 끝났다면, 이제 컴파일러가 제대로 동작하는지 확인해야 한다.

그 가장 쉬운 방법은 C 프로그램 하나를 직접 작성해보는 것이다.

이미 최소한 하나 이상의 프로그래밍 언어를 알고 있을 테니, 작지만 내용을 꽤 많이 다음 예제로 시작해도 괜찮다.

[1-1] ex1.c

```
#include <stdio.h>

/* This is a comment.
 */
int main(int argc, char *argv[])
{
    int distance = 100;

    // this is also a comment
    printf("You are %d miles away.\n", distance);

    return 0;
}
```

[1-1-1] 코드 분석(Breaking It Down)

이 코드에는 C 언어의 몇 가지 특징이 들어 있다. 타이핑하면서 이미 눈채했을 수도 있고 아닐 수도 있다. 이제 한 줄씩 빠르게 설명하겠다.

지금 당장 모든 걸 이해하지 못해도 괜찮다. 이건 C에 대한 빠른 맛보기일 뿐이고, 이 책의 나머지 부분에서 전부 다시 배우게 된다.

[1-1-2] 줄별 설명

{1} #include

다른 파일의 내용을 이 소스 파일로 가져오는 방법이다.

C에서는 보통 .h 확장자를 가진 파일을 헤더 파일이라 부르며, 프로그램에서 사용할 함수들의 목록이 들어 있다.

{2} /**/

여러 줄 주석이며, 이 사이에 원하는 만큼 여러 줄의 텍스트를 넣을 수 있다.

{3} int main(int argc, char *argv[])

지금까지 사용해온 main 함수의 좀 더 완전한 형태이다. C 프로그램은 운영체제가 프로그램을 로드한 뒤, 이름이 main 인 함수를 실행하는 방식으로 동작한다.

이 함수는 완전한 형태로 쓰려면

- int 를 반환해야 하고
- 두 개의 매개변수를 받아야 한다.
 - 인자의 개수를 나타내는 int
 - 문자열 배열인 char *argv[]

방금 설명이 머리 위로 날아갔다고 느껴도 괜찮다. 곧 다시 자세히 다룰 것이다.

{4} 중괄호 {}

함수의 본문을 시작할 때 { 문자를 쓴다. 이것은 블록의 시작을 의미한다.

Python에서는 : 와 들여쓰기를 사용했고, 다른 언어에서는 begin 이나 do 같은 키워드를 쓰기도 한다.

마지막으로 }를 써서 main 함수를 끝낸다. 이것으로 프로그램 전체가 끝난다.

{5} 변수 선언과 초기화

변수를 선언하면서 동시에 값을 대입하는 코드다.

형식은 다음과 같다.

타입 변수이름 = 값;

C에서는 (제어문을 제외한) 대부분의 문장이 ;(세미콜론)으로 끝난다.

{6} //

또 다른 형태의 주석이다.

Python 이나 Ruby 처럼 // 부터 줄 끝까지 주석이 된다.

{7} printf()

많은 언어와 마찬가지로 함수 호출은 다음과 같은 형식을 가진다.

함수이름(인자 1, 인자 2);

인자는 없을 수도 있고, 여러 개일 수도 있다.

printf는 특히 여러 개의 인자를 받을 수 있는 특이한 함수인데, 이 부분은 나중에 다시 설명한다.

```
{8} return 0;
```

main 함수에서 값으로 반환하는 부분이다.

이 값은 운영체제(OS)에 전달하는 종료 코드다.

UNIX 계열에서 이 값이 어떻게 사용되는지는 나중에 따로 다룰 것이다.

[1-1-3] 실행 결과

이 코드를 ex1.c 파일로 저장한 뒤, 아래와 같은 명령을 실행하면 된다.

```
$ make ex1
cc -Wall -g ex1.c -o ex1
$ ./ex1
You are 100 miles away.
$
```

첫 번째 명령인 make 는 C 프로그램(그리고 다른 많은 프로그래밍 언어)을 빌드하는 도구다.

- -Wall: 경고 최대한 다 보기
- -g: 디버깅 정보 포함(lldb/gdb 용)
- ex1.c: 소스 파일
- -o ex1: 결과 실행 파일 이름

make ex1 을 실행하면:

- ex1.c 파일을 찾고
- 컴파일러를 실행해 결과를 ex1 이라는 파일로 만든다.

이 ex1 파일은 실행 파일이며, ./ex1 처럼 실행하면 프로그램의 출력 결과를 볼 수 있다.

[1-2] 망가뜨리는 방법(How to Break It)

이 책에서는 각 프로그램마다. 가능하다면 그 프로그램을 어떻게 망가뜨릴 수 있는지를 알려주는 작은 섹션이 있다. 프로그램에 이상한 입력을 주거나, 이상한 방식으로 실행하건, 코드를 바꿔서 크래시나 컴파일 에러를 직접 보게 할 것이다.

이 프로그램에서는, 그냥 아무 요소나 무작위로 제거해 보면서도 컴파일이 되는지 시도해보면 된다. 무엇을 지울 수 있을지 추측해서 지워보고, 다시 컴파일한 다음, 어떤 에러가 나는지 확인해보라.

[1-3] 추가 과제(Extra Credit)

- 텍스트 편집기로 ex1 파일을 열고, 임의의 부분을 수정하거나 삭제해보자. 실행해보고 어떤 일이 일어나는지 확인하자.

- 다섯 줄 이상의 텍스트를 출력해보거나, “hello world” 보다 조금 더 복잡한 내용을 출력해보라.

- `man 3 printf` 를 실행해서 이 함수와 다른 많은 함수들에 대해 읽어보자.

- 각 줄마다 이해되지 않은 기호(symbol)를 적어보고, 그 의미를 추측해보자. 종이에 작은 표를 만들어서 네가 추측한 내용을 적어두고, 나중에 맞았는지 확인해보자.

Exercise 2. Makefiles 를 사용한 빌드

이번에는 make 라는 프로그램을 사용해서 실습 코드를 빌드하는 과정을 단순화할 것이다. make 프로그램은 아주 오래전부터 존재해 왔고, 그만큼 다양한 종류의 소프트웨어를 빌드하는 방법을 알고 있다.

이 실습에서는 이후 과정을 따라갈 수 있을 정도의 Makefile 문법만 가르칠 것이고, 나중의 실습에서 더 완전한 Makefile 사용 법을 다룰 것이다.

[2-1] Make 사용하기

make 의 동작 방식은 의존성(dependency)을 선언하고, 그것을 어떻게 빌드할지 설명하거나, 대부분의 일반적인 소프트웨어를 빌드하는 방법에 대한 make 의 내부 지식을 사용하는 것이다. make 는 수십 년 동안 다양한 파일을 다른 파일로부터 빌드하는 방법을 알고 있다.

이전 실습에서 이미 다음과 같은 명령을 사용해 봤다.

```
$ make ex1
# 또는
$ CFLAGS="-Wall" make ex1
```

첫 번째 명령에서 make 에게 이렇게 말하는 것이다.
“ex1 이라는 파일을 만들고 싶다.”

그러면 make 는 다음과 같은 과정을 거친다.

1. ex1 파일이 이미 존재하는가?
2. 아니오 그렇다면 ex1 로 시작하는 다른 파일이 있는가?
3. 예. ex1.c 라는 파일이 있다. .c 파일을 빌드하는 방법을 알고 있는가?
4. 예. cc ex1.c -o ex1 명령을 실행한다.
5. cc 를 사용해 ex1.c 로부터 ex1 을 만든다.

위의 두 번째 명령은 make 에 옵션을 전달하는 방법이다. UNIX 셸에 익숙하지 않다면, 이런식으로 프로그램이 사용할 환경 변수를 만들 수 있다는 점을 알아두면 된다.

보통은 셸에 따라 다음과 같이 쓰기도 한다.

```
export CFLAGS="-Wall"
```

하지만 실행할 명령 앞에 바로 환경 변수를 붙이면, 그 명령이 실행되는 동안에만 해당 변수가 적용된다.

이 예제에서는:

```
CFLAGS="-Wall" make ex1
```

을 사용했는데, 이는 make 가 내부적으로 실행하는 cc 명령에 -Wall 옵션을 추가하라는 의미다. -Wall 옵션은 컴파일러에게 모든 경고를 출력하라고 지시한다. (아이러니하게도 실제로는 모든 경고를 다 출력하지는 않는다.)

이 방식만으로도 make 를 꽤 잘 사용할 수 있지만, 이제 Makefile 을 만들어서 make 를 조금 더 잘 이해해 보자.

우선 다음 내용만 들어 있는 파일을 하나 만든다.

[2-2] ex2.1.mak

```
CFLAGS=-Wall -g
```

```
clean:
```

```
    rm -f ex1
```

이 파일을 현재 디렉터리에 Makefile 이라는 이름으로 저장한다.

make 는 자동으로 Makefile 이라는 파일이 있다고 가정하고 실행한다.

경고: TAB 문자만 사용해야 하며, TAB 과 스페이스를 섞어 쓰면 안된다.

이 Makefile 에서는 새로운 내용이 몇 가지 있다.

먼저 CFLAGS 를 파일 안에서 설정했기 때문에, 다시 설정할 필요가 없고 디버깅을 위한 -g 옵션도 추가했다.

그 다음 clean 이라는 섹션이 있는데, 이건 프로젝트를 정리(clean up)하는 방법을 make 에게 알려준다.

ex1.c 파일과 같은 디렉터리에 Makefile 이 있는지 확인한 후, 다음 명령을 실행한다.

```
$ make clean
```

```
$ make ex1
```

[2-3] 실행 결과

정상적으로 동작했다면 다음과 같은 결과를 보게 될 것이다.

```
$ make clean
rm -f ex1

$ make ex1
cc -Wall -g ex1.c -o ex1
ex1.c: In function 'main':
ex1.c:3: warning: implicit declaration of function 'puts'
$
```

여기서 `make clean` 을 실행하면, `make` 가 우리가 정의한 `clean` 타겟을 실행한다. `Makefile` 을 다시 보면, 이 명령 아래에 들여쓰기한 후 `make` 가 실행할 셸 명령을 적어 둔 것을 볼 수 있다.

이 안에는 원하는 만큼 명령을 넣을 수 있기 때문에, `make` 는 매우 훌륭한 자동화 도구다.

경고:

만약 `ex1.c` 에 `#include <stdio.h>` 를 추가했다면, `puts` 에 대한 경고(사실상 에러여야 하는)가 출력되지 않을 것이다. 여기서는 일부러 고치지 않았기 때문에 경고가 나온다.

`Makefile` 에 `ex1` 을 직접 명시하지 않았는데도, `make` 는 여전히 `ex1` 을 빌드하는 방법을 알고 있으며 우리가 설정한 옵션도 사용한다는 점에 주의하자.

[2-4] 일부러 망가뜨려 보기

이제 `Makefile` 이 어떻게 깨지는지 보기 위해 일부러 문제를 만들어 보자.
`rm -f ex1` 줄에서 들여쓰기(tab)를 제거하고 다시 `make clean` 을 실행해보자.

그러면 다음과 같은 메시지가 나올 것이다.

```
$ make clean
Makefile:4: *** missing separator. Stop.
```

항상 들여쓰기를 잊지 말 것.

이런 이상한 에러가 나오면, TAB 문자를 일관되게 사용했는지 꼭 확인하자. 일부 `make` 구현체는 굉장히 엄격하다.

[2 -4-1] Make & Makefile 핵심 요약

[목적]

- C 프로그램 컴파일을 간단하게 자동화하기 위해 make 와 Makefile 을 사용한다.
- 긴 컴파일 명령을 매번 입력하지 않아도 되게 한다.

[make 란?]

- 빌드 자동화 도구
- 만들고 싶은 파일 이름만 주면, 필요한 규칙을 찾아 알아서 컴파일한다.

[예시]

```
make ex1
```

- ex1 실행 파일이 없으면
- ex1.c 를 찾고
- 내부 규칙으로 아래 명령을 실행한다.

```
cc ex1.c -o ex1
```

[CFLAGS]

- 컴파일 옵션을 담는 변수

```
CFLAGS = -Wall -g
```

- -Wall: 모든 경고 출력
- -g: 디버깅 정보 포함
- make 가 컴파일할 때 자동으로 사용

[Makefile 기본 구조]

```
CFLAGS = -Wall -g
```

```
clean:
```

```
    rm -f ex1
```

구성 요소 설명

- CFLAGS: 컴파일 옵션 설정
- clean: 타겟(target) 이름
- rm -f ex1: 실행할 셸 명령

주의

- 명령 줄 앞은 반드시 TAB 키 사용(스페이스 x)

[자주 사용하는 명령어]

```
make clean    #실행파일 삭제  
make ex1      #ex1.c -> ex1 컴파일
```

[make clean 의 의미]

- 빌드 결과물(실행 파일)을 지우고
- 처음부터 다시 빌드할 수 있게 초기화

[자주 나는 에러]

*** missing separator. Stop.

- 원인: TAB 대신 스페이스 사용
- 해결: 명령 줄 앞을 TAB 으로 수정

[2-5] 추가 과제(Extra Credit)

- make 명령만으로 ex1 을 빌드할 수 있도록 all:ex1 타겟을 생성하시오.
- man make 를 읽고 make 를 실행하는 방법에 대해 더 알아보시오
- man gcc 를 읽고 -Wall 플래그가 무엇을 의미하는지 알아보시오
- 온라인에서 Makefile 에 대해 조사하고, 이 Makefile 을 더 개선할 수 있는지 살펴보세요.
- 다른 C 프로젝트에서 Makefile 을 하나 찾아서, 그것이 무엇을 하는지 이해해보시오.

Exercise 3. 서식화된 출력

Makefile 은 계속 가지고 있자. 에러를 찾는데 도움이 되고, 나중에 더 많은 작업을 자동화해야 할 때 거기에 내용을 추가하게 될 것이다. 많은 프로그래밍 언어들이 C 방식의 출력 포맷을 사용하므로, 이제 그걸 직접 사용해 본다.

[3-1] ex3.c

```
#include <stdio.h>

int main()
{
    int age = 10;
    int height = 72;

    printf("I am %d years old.\n", age);
    printf("I am %d inches tall.\n", height);

    return 0;
}
```

이 코드를 다 작성했다면, 평소처럼 `make ex3` 를 실행해서 빌드하고 실행한다. 그리고 모든 경고(warning)를 반드시 수정해야 한다.

이 연습문제는 아주 적은 코드 안에 많은 내용이 들어 있으므로, 하나씩 분해해서 살펴본다.

- 먼저 `stdio.h` 라는 또 다른 헤더 파일을 포함한다. 이것은 컴파일러에게 표준 입출력(Input/Output) 함수를 사용할 것이라고 알려준다.
- 다음으로 `age` 라는 변수를 사용하고, 값을 10 으로 설정한다.
- 그 다음 `height` 라는 변수를 사용하고, 값을 72 로 설정한다.
- 그리고 `printf` 함수를 사용해 지구에서 가장 큰 10 살 아이의 나이와 키를 출력한다.
- `printf` 안을 보면, 다른 많은 언어들에서 보았을 포맷 문자열(format string)을 사용하고 있는 것을 볼 수 있다.
- 이 포맷 문자열 뒤에 `printf` 가 포맷 문자열 안에 “대체해서 넣을 변수들을 전달한다.

결과적으로 `printf` 는 변수들을 받아서 새로운 문자열을 구성한 뒤, 그 문자열을 터미널에 출력한다.

[3-1] 결과로 보여야 할 것(What You Shoud See)

전체 빌드를 실행하면, 다음과 비슷한 결과가 나와야 한다.

```
$ make ex3
cc -Wall -g ex3.c -o ex3
$ ./ex3
I am 10 years old.
I am 72 inches tall.
$
```

곧 나는 make 를 실행하라고 계속 말해주지 않을 것이므로, 지금 단계에서 제대로 동작하는지 반드시 확인하자.

[3-2] 외부 조사(External Research)

각 연습문제의 추가 과제(Extra Credit) 섹션에서는 스스로 정보를 찾아보도록 요구할 것이다. 이것은 자립적인 프로그래머가 되기 위해 매우 중요한 부분이다. 문제가 생길 때마다 누구에게 질문만 한다면, 스스로 문제를 해결하는 방법을 절대 배우지 못한다.

그렇게 되면 자신의 실력에 대한 자신감도 쌓이지 않고, 항상 누군가가 대신 일을 해줘야만 하게 된다. 이 습관을 깨는 방법은 간단하다. 먼저 스스로 답을 생각해보고, 그 다음에 그 답이 맞는지 확인하는 것이다.

이를 위해 프로그램을 일부러 망가뜨려 보고, 실험해 보고, 스스로 자료를 조사해야 한다.

이 연습문제에서는 온라인에 들어가서 printf 의 모든 escape 코드와 format 시퀀스를 찾아보기를 바란다.

- escape 코드는 Wn, Wt 같은 것으로 각각 줄바꿈, 탭을 출력한다.
- format 시퀀스는, %s, %d 같은 것으로 문자열이나 정수를 출력하게 해준다.

이것들을 전부 찾아보고, 어떻게 수정할 수 있는지, 그리고 어떤 정밀도(precision)와 폭(width)을 줄 수 있는지도 확인해 보자. 앞으로 이런 종류의 과제는 모두 추가 과제(Extra Credit) 섹션에 들어가게 될 것이며, 반드시 수행해야 한다.

[3-3] 망가뜨려 보기(How to Break It)

다음 방법들로 이 프로그램을 일부러 망가뜨려 보자. 컴퓨터에 따라 크래시가 날 수도 있고, 안 날 수도 있다.

- 첫 번째 printf 호출에서 age 변수를 제거한 뒤 다시 컴파일해라. 몇 개의 경고가 발생할 것이다.

```
$make ex3
cc -Wall -g    ex3.c    -o ex3
ex3.c:14:19: warning: more '%' conversions than data arguments [-Wformat-insufficient-args]
    14 |     printf("I am %d years old\n");
        |                      ~^
ex3.c:11:9: warning: unused variable 'age' [-Wunused-variable]
    11 |     int age = 10;
        |         ^~~
2 warnings generated.
```

- 이 새 프로그램을 실행하면, 크래시가 나거나 아주 이상한 나이가 출력될 것이다.

```
$/ex3
I am -16 years old
I am 72 inches tall
```

- printf를 다시 원래대로 돌린 다음, age를 초기화되지 않도록 int age = 10;을 int age;로 바꿔라. 그리고 다시 빌드하고 실행해봐라.

```
$/ex3
I am -16 years old
I am 72 inches tall
```


[3-4] 추가 과제(Extra Credit)

- ex3.c 를 망가뜨릴 수 있는 다른 방법들을 최대한 많이 찾아라.
- man 3 printf 를 실행해서 사용할 수 있는 다른 % 포맷 문자들에 대해 읽어라.(다른 언어를 써봤다면 익숙할 것이다. 이들은 printf 에서 유래했다.)

The format string is composed of zero or more directives: ordinary characters (not %), which are copied unchanged to the output stream; and conversion specifications, each of which results in fetching zero or more subsequent arguments. Each conversion specification is introduced by the % character. The arguments must correspond properly (after type promotion) with the conversion specifier. After the %, the following appear in sequence:

- Makefile 의 all 목록에 ex3 를 추가하라. 지금까지의 모든 연습문제를 한 번에 빌드하라.
- Makefile 의 clean 목록에도 ex3 를 추가하라. 필요할 때 make clean 으로 제거할 수 있도록 하라.

```
CC=<컴파일러>
CFLAGS=<컴파일 옵션>
LDFLAGS=<링크 옵션>
LDLIBS=<링크 라이브러리 목록>
OBJS=<오브젝트 파일 목록>
TARGET=<빌드 대상>
```

```
$(TARGET): $(OBJS)
    $(CC) -o $@ $(OBJS)
```

```
all: $(TARGET)
```

```
clean:
```

```
    rm -f *.o
    rm -f $(TARGET)
```

```
CFLAGS=-Wall -g
```

```
TARGET=ex3 ex1
```

```
all: $(TARGET)
```

```
clean:
```

```
    rm -f *.o $(TARGET)
```

```

CFLAGS=-Wall -g
TARGET=ex3 ex1

all: $(TARGET)

ex1: ex1.c
    cc $(CFLGS) ex1.c -o ex1

ex3: ex3.c
    cc $(CFLGS) ex3.c -o ex3

clean:
    rm -f *.o $(TARGET)

```

[Make 에서 규칙을 쓰지 않아도 빌드가 되는 이유]

ex1: ex1.c 같은 규칙을 Makefile 에 명시하지 않았는데도 make ex1., make ex3 가 정상 동작하는 이유는 Make 에 내장된 자동 규칙(built-in rules) 때문이다. Make 는 C 프로젝트를 매우 흔하게 다루기 때문에, .c 파일과 동일한 이름의 실행 파일을 만드는 기본 패턴 규칙을 이미 알고 있다. 그래서 ex3.c 가 존재하면, Make 는 이를 입력 파일로 삼아 ex3 실행 파일을 생성할 수 있다고 판단하고 자동으로 컴파일을 수행한다.

[Make 가 내부적으로 사용하는 개념적 규칙]

개념적으로 Make 는 다음과 같은 패턴 규칙을 내장하고 있다.

- %.c 파일이 있으면 cc 를 사용해 %이름의 실행 파일을 만든다.
- 이때 컴파일 옵션으로 \$(CFLAGS)와 \$(CPPFLAGS)를 함께 사용한다.

이 때문에 사용자가 별도의 규칙을 작성하지 않아도 make ex3 명령은 내부적으로 cc ex3.c -o ex3 와 같은 동작을 수행한다.

[CFLAGS 가 자동으로 적용되는 이유]

CFLAGS 는 Make 가 표준으로 약속해 둔 변수 중 하나다. 내장된 C 컴파일 규칙은 \$(CFLASG)를 자동으로 참조하도록 정의되어 있기 때문에, Makefile 에 CFLAGS=-Wall -g 만 작성해 두어도 자동 규칙을 통해 컴파일할 때 해당 옵션이 자연스럽게 적용된다. 즉, 사용자가 명시적으로 cc \$(CFLAGS)… 를 작성하지 않아도 동일한 효과가 나타난다.

[언제 자동 규칙에만 의존해도 되는가]

자동 규칙은 다음과 같은 단순한 경우에 매우 잘 맞는다.

- 하나의 .c 파일로 하나의 실행 파일을 만드는 경우
- 출력 파일 이름이 소스 파일 이름과 동일한 경우
- 추가 라이브러리나 특수한 링크 옵션이 필요 없는 경우

이런 사항에서는 Makefiledp 에 all:ex1 정도만 있어도 충분한 빌드가 가능하다.

[언제 명시적인 규칙이 필요한가]

반대로 다음과 같은 경우에는 자동 규칙만으로 부족하다.

- 여러 개의 .c 파일을 링크해야 하는 경우
- -lm, -lpthread 같은 라이브러리 옵션이 필요한 경우
- 실행 파일 이름이 소스 파일 이름과 다른 경우
- 타깃마다 다른 컴파일 옵션을 사용해야 하는 경우

이럴 때는 명시적으로 규칙을 작성해야 하며, 가독성과 유지보수를 위해서라도 실제 프로젝트에서는 명시 규칙을 사용하는 경우가 많다.

[정리]

Make 에서 규칙을 작성하지 않았는데도 빌드가 되는 것은 오류가 아니라, Make 의 자동 규칙 덕분에 의도된 정상 동작이다. 이 동작 원리를 이해하고 있으면, 단순한 과제에서는 Makefile 을 간결하게 작성할 수 있고, 규모가 커질 경우 언제 명시적인 규칙이 필요한지도 판단할 수 있다.

Exercise 4. Using a Debugger (GDB, LLDB)

이 장의 목적은 C 프로그램을 실행하면서 발생하는 오류를 보다 체계적으로 분석하고 해결하기 위해 디버거(Debugger)를 사용하는 방법을 익히는 것이다. C 언어는 실행 중 오류가 발생해도 원인을 명확하게 알려주지 않는 경우가 많기 때문에, 프로그램 내부 상태를 직접 확인할 수 있는 도구가 필수적이다. 이 장에서는 GNU Debugger, 즉 GDB를 중심으로 디버깅의 기본 개념과 사용 방법을 다룬다.

[4-1] GDB 기초 설명

[1] GDB 란 무엇인가

GDB(GNU Debugger)는 프로그램을 실행하면서 특정 시점에서 멈추게 하거나, 한 줄씩 실행하면서 변수 값과 함수 호출 흐름을 확인할 수 있는 디버깅 도구이다. 이를 통해 프로그램이 언제, 어디서, 어떤 이유로 잘못된 동작을 하는지 추적할 수 있다. 단순히 실행 결과만 보는 것이 아니라, 실행 과정 자체를 관찰할 수 있다는 점이 GDB의 가장 큰 장점이다.

[2] 디버깅이 필요한 이유

C 프로그램은 컴파일이 성공하더라도 실행 중에 갑자기 종료되거나(segmentation fault), 예상과 다른 결과를 출력하는 경우가 자주 발생한다. 이런 문제는 printf만으로는 원인을 찾기 어렵다. 디버거를 사용하면 프로그램의 실행 흐름을 제어하고, 각 시점에서 변수의 값과 함수 호출 상태를 확인함으로써 오류의 근본 원인을 정확히 파악할 수 있다.

[3] 디버깅을 위한 컴파일

GDB를 사용하기 위해서는 컴파일 시 반드시 디버그 정보를 포함해야 한다. 이를 위해 -g 옵션을 사용한다.

```
gcc -Wall -g ex4.c -o ex4
```

이때 -Wall은 컴파일 경고를 모두 출력하도록 하고, -g는 GDB가 소스 코드 정보를 읽을 수 있도록 해준다. 이 옵션이 없으면 디버깅 시 소스 코드와 변수 정보를 제대로 확인할 수 없다.

[4-2] GDB 사용 설명

[1] GDB 실행과 기본 사용

컴파일된 프로그램을 GDB로 실행하려면 다음 명령을 사용한다.

```
gdb ./ex4
```

GDB가 실행되면 (gdb)프롬프트가 나타난다. 여기서 run 명령을 사용하면 프로그램이 실행된다. 이 단계에서는 아직 일반 실행과 큰 차이가 없지만, 이후 브레이크포인트를 설정하면 디버깅이 본격적으로 시작된다.

[2] 브레이크포인트와 실행 제어

브레이크포인트는 프로그램 실행을 특정 지점에서 멈추게 하는 기능이다. 예를 들어, main 함수 시작 지점에서 멈추게 하려면 다음과 같이 입력한다.

```
break main
```

이후 run 을 실행하면 프로그램은 main 함수에 진입하자마자 멈춘다. 이 상태에서 개발자는 프로그램이 실행되기 직전의 내부 상태를 확인할 수 있다.

[3] 한 줄씩 실행하며 흐름 확인

프로그램이 멈춘 상태에서는 한 줄씩 실행할 수 있다. next 명령은 현재 줄을 실행하고 다음 줄로 이동하되, 함수 호출은 내부로 들어가지 않고 건너뛴다. 반면 step 명령은 함수 호출 내부로 들어가 한 줄씩 실행한다. 이를 통해 코드의 실행 흐름을 매우 세밀하게 추적할 수 있다.

[4] 변수 값 확인

GDB에서는 실행 중인 시점의 변수 값을 직접 확인할 수 있다. print 변수명 또는 줄여서 **p 변수명** 명령을 사용하면 된다. 이 기능은 값이 언제, 어디서 잘못 바뀌었는지를 추적하는 데 매우 유용하다.

[5] 프로그램이 죽었을 때: Backtrace

프로그램이 실행 중 오류로 종료되었을 경우, backtrace 또는 bt 명령을 사용하면 함수 호출 스택을 확인할 수 있다. 이를 통해 어떤 함수가 어떤 순서로 호출되다가 문제가 발생했는지를 알 수 있으며, 실제 디버깅 과정에서 가장 자주 사용되는 핵심 기능 중 하나이다.

[6] 디버깅 종료

디버깅을 마치고 GDB 를 종료하면 quit 명령을 사용한다. 필요에 따라 다시 GDB 를 실행해 다른 브레이크포인트를 설정하거나 새로운 문제를 분석할 수 있다.

[4-3] LLDB 기초 설명

macOS(OS X) 환경에서는 기본적으로 GDB 가 제공되지 않으며, 대신 LLDB(Debugger)라는 디버깅 도구를 사용한다. LLDB 는 LLVM 프로젝트의 일부로, Xcode 와 함께 기본 설치되는 공식 디버거다. 기능과 개념은 GDB 와 거의 동일하지만, 명령어 문법이 약간 다르다. 따라서 GDB 를 이해했다면 LLDB 도 어렵지 않게 사용할 수 있다.

[1] LLDB 실행과 프로그램 시작

LLDB 는 터미널에서 다음과 같이 실행한다.

```
lldb ./ex4
```

LLDB 가 실행된후 run [args] 명령을 사용하면 프로그램이 실행된다. 이때 [args]에는 프로그램 실행 시 전달할 인자를 넣을 수 있다. 기본적인 사용 방식은 GDB 의 run 명령과 동일하다.

[2] 브레이크포인트 설정

LLDB 에서 브레이크포인트를 설정하려면 다음과 같은 명령을 사용한다.

```
breakpoint set --name main
```

이 명령은 특정 함수 이름(main 등)에 브레이크포인트를 설정한다. 하지만 실제로는 b 라는 축약 명령을 더 많이 사용하며, 이 방식이 훨씬 간단하다.

```
b main
```

브레이크포인트를 설정한 후 run 을 실행하면 프로그램은 해당 지점에서 멈추게 된다.

[3] 실행 흐름 추적

프로그램이 멈춘 상태에서는 한 줄씩 실행하며 흐름을 추적할 수 있다. next 명령은 현재 줄을 실행하고 다음 줄로 이동하되, 함수 호출은 내부로 들어가지 않는다. 반면 step 명령은 함수 호출 내부로 들어가 한 줄씩 실행한다. 이 두 명령을 적절히 사용하면 코드의 실행 경로를 매우 정확하게 분석할 수 있다.

[4] 변수 값 확인

LLDB 에서는 실행 중인 프로그램의 변수 값을 확인할 수 있다. 이를 위해 print expr 또는 축약형 p expr 명령을 사용한다. 이 기능을 통해 특정 시점에서 변수 값이 예상과 다른지 확인할 수 있으며, 논리 오류나 메모리 관련 무제를 추적하는 데 매우 중요하다.

[5] Backtrace(호출 스택 확인)

프로그램이 오류로 종료되었거나 현재 실행 위치를 파악하고 싶을 때는 thread backtrace 명령을 사용한다. 축약형 bt 이다. 이 명령은 현재 함수가 어떤 함수들에 의해 호출되었는지를 역순으로 보여준다. 복잡한 함수 호출 구조에서 오류 발생 지점을 찾을 때 핵심적인 기능이다.

[6] 실행 재개 및 종료

브레이크포인트에서 멈춘 프로그램을 다시 실행하려면 continue 또는 축약형 c 명령을 사용한다. 디버깅을 종료하고 LLDB 를 빠져나오려면 quit 명령을 사용한다.

[7] 도움말과 셸 기능

LLDB 에서는 help 명령을 통해 전체 명령어 목록을 확인할 수 있으며, 특정 명령어에 대한 자세한 설명도 볼 수 있다. 또한 shell 명령을 사용하면 디버깅을 종료하지 않고 임시로 셸을 실행할 수 있어 파일 확인이나 빌드 명령(make) 등을 수행할 수 있다.

[8] 브레이크포인트 관리

설정된 브레이크포인트를 제거하려면 `clear` 명령을 사용한다. 현재 설정된 브레이크포인트나 감시점(watchpoint)정보를 확인하려면 `info break` 또는 `info watch` 명령을 사용한다. 이를 통해 디버깅 중 브레이크포인트 상태를 관리할 수 있다.

[9] 실행 중인 프로세스에 붙기

이미 실행 중인 프로그램을 디버깅해야 할 경우, `attach -p pid` 명령을 사용하여 해당 프로세스에 연결할 수 있다. 디버깅을 마친 후에는 `detach` 명령으로 프로세스에서 분리할 수 있다. 이 기능은 서버 프로그램이나 백그라운드 프로세스를 분석할 때 유용하다.

[10] 소스 코드 확인

`list` 명령을 사용하면 현재 실행 중인 위치 주변의 소스 코드를 확인할 수 있다. 기본적으로 다음 10 줄을 보여주며, `list -`를 사용하면 이전 10 줄을 확인할 수 있다. 이는 코드 흐름을 눈으로 확인하면서 디버깅할 때 매우 유용하다.

[4-4] LLDB 기본 명령어

(lldb) run	# 실행
(lldb) bt	# 백트레이스
(lldb) break main	# 브레이크포인트
(lldb) next	# 다음 줄
(lldb) step	# 함수 안으로
(lldb) quit	

Exercise 5. C 연산자 암기하기

이 장은 C 언어의 연산자와 기본 문법을 “외워서” 익히자는 철학을 설명하는 부분이다. 저자는 처음 프로그래밍을 배울 때 흔히 하는 방식 - 이해가 완전하지 않은 코드를 따라 치고, 시행착오로 감을 잡는 방식 - 이 초보자에게는 효과적이지만, 이미 다른 언어를 하나라도 알고 있는 사람에게는 비효율적이라고 말한다.

[5-1] 왜 암기가 필요한가

이미 하나 이상의 프로그래밍 언어를 알고 있다면, 문법을 “자연스럽게 몸에 베게 하는 방식”은 가능하긴 하지만 시간이 오래 걸린다. 대신 저자는 기본 기호와 문법을 먼저 명확하게 암기한 뒤, 이를 연습 문제로 적용하는 방식이 훨씬 빠르고 효율적이라고 주장한다.

이 방식은 외국어 학습과 비슷하다. 단어와 문법을 먼저 외우고, 그다음 실제 대화에서 사용하는 것처럼, C 언어의 기호·연산자·문법을 먼저 머릿속에 넣어두면 코드를 읽고 쓰는 속도가 훨씬 빨라진다.

[5-2] 암기에 대한 오해에 대한 경구

저자는 “암기는 창의성을 떨어뜨린다”는 주장에 강하게 반대한다. 암기를 싫어하는 사람들이 흔히 말하는 이런 주장은 근거가 없고, 오히려 효율적인 학습을 방해한다고 말한다.

저자 본인은 그림을 그리고, 기타를 만들고, 노래하고, 코딩하고, 책을 쓰는 사람이며, 동시에 많은 것을 암기해왔다고 말한다. 즉, 암기와 창의성은 전혀 반대되는 개념이 아니라는 것이다.

[5-3] C 문법을 암기하는 구체적인 방법

저자가 추천하는 암기 방법은 다음과 같은 단계로 이루어진다.

첫째, 플래시카드를 만든다. 카드 한쪽에는 C의 기호(예: +, ==, *, &&)를 쓰고, 다른 쪽에는 그 의미와 설명을 적는다. 컴퓨터 프로그램(예: Anki)을 써도 되지만, 저자는 직접 카드를 만드는 과정 자체가 암기에 도움이 된다고 말한다.

둘째, 카드를 섞은 뒤 한쪽 면만 보고 다른 쪽 내용을 떠올리려고 노력한다.

셋째, 기억이 안나면 바로 답을 확인하고, 그 카드는 따로 “실패한 카드 더미”에 넣는다.

넷째, 모든 카드를 한 번 다 본 뒤, 틀린 카드들만 다시 집중적으로 반복한다.

다섯째, 15~30 분 정도의 암기 시간이 끝나면 끝까지 외워지지 않는 카드들이 남게 되는데, 이 카드들은 들고 다니면서 자투리 시간에 계속 반복해서 본다.

이 방법의 핵심은 즉각적인 회상이다. C 언어의 연산자, 키워드, 문법은 코드를 작성하거나 읽을 때 생각하지 않고 바로 떠올라야 한다. 매번 찾아보거나 헛갈리면 흐름이 계속 끊기기 때문이다. 그래서 이 플래시카드 방식은 “아는 것”이 아니라 “즉시 쓸 수 있는 것”으로 만드는 데 최적화된 방법이다.

[5-4] 암기는 양방향으로 해야 한다.

중요한 점은 플래시 카드를 한 방향으로만 외우면 안 된다는 것이다.

- 기호를 보고 의미를 말할 수 있어야 하고
- 설명을 보고 어떤 기호인지 떠올릴 수 있어야 한다.

이 두 가지 모두 되어야 실제 코딩에서 자유롭게 사용할 수 있다.

마지막으로 저자는, 이 암기 작업을 하면서 코딩 연습을 멈출 필요는 없다고 말한다. 암기와 실습을 병행해야 하고, 바로 다음 연습 문제에서 이 암기한 내용들을 실제 코드로 사용하게 될 것이라고 예고한다.

[5-5] 연산자 목록(*플래시카드)

가장 처음에 나오는 연산자들은 산술 연산자로, 거의 모든 다른 프로그래밍 언어와 매우 비슷하다. 카드를 만들 때, 설명 쪽에는 이것이 산술 연산자라는 것과 무엇을 하는지 적어야 한다.

연산자	의미	예시
+	덧셈	$a + b$
-	뺄셈	$a - b$
*	곱셈	$a * b$
/	나눗셈	a / b
%	나머지	$a \% b$

관계 연산자는 값들이 서로 같은지 여부를 테스트한다. 이 역시 프로그래밍 언어 전반에서 매우 흔하다.

연산자	의미	예시
==	같다	$a == b$
!=	다르다	$a != b$
>	크다	$a > b$
<	작다	$a < b$
>=	크거나 같다	$a >= b$
<=	작거나 같다	$a <= b$

논리 연산자는 논리적인 조건 테스트를 수행한다. 이 연산자들이 무엇을 하는지 이미 알고 있을 것이다.

연산자	의미	예시
&&	논리 AND	a && b
	논리 OR	a b
!	논리 NOT	!a

유일하게 조금 특이한 것은 논리 삼항 연산자인데, 이것은 이 책의 뒤쪽에서 배우게 될 것이다.

연산자	의미	예시
?:	조건 연산자	a > b ? a : b

비트 연산자는 현대적인 코드에서는 자주 접하지 않을 수도 있는 작업을 수행한다. 이 연산자들은 바이트나 다른 데이터 타입을 구성하는 비트들을 다양한 방식으로 변경한다. 나는 이책에서 이것들을 자세히 다루지는 않을 것이지만, 특정한 저수준 시스템을 다룰 때는 매우 유용하다.

연산자	의미	예시
&	비트 AND	a & b
	비트	a b
^	비트 XOR	a ^ b
~	비트 NOT	~a
<<	왼쪽 시프트	a << 1
>>	오른쪽 시프트	a >> 1

대입 연산자는 단순히 표현식을 변수에 할당한다. 하지만 C에서는 많은 다른 연산자들이 대입 연산자와 결합되어 있다. 그래서 내가 “and-equal”이라고 말할 때는 논리 연산자가 아니라 비트 연산자를 의미하는 것이다.

연산자	의미	예시
=	대입	a = 10
+=	더해서 대입	a += 5
-=	빼서 대입	a -= 5
*=	곱해서 대입	a *= 5
/=	나눠서 대입	a /= 5
%=	나머지 대입	a %= 5
&=	비트 AND 대입	a &= b
=	비트 OR 대입	a = b
^=	비트 XOR 대입	a ^= b
<<=	왼쪽 시프트 대입	a <<= 1
>>=	오른쪽 시프트 대입	a >>= 1

나는 이것들을 데이터 연산자라고 부르고 있지만, 실제로는 포인터의 특성, 멤버 접근, 그리고 C에서의 다양한 데이터 구조 요소들이 관련되어 있다.

연산자	의미	예시
*	포인터 역참조	*p
&	주소 연산자	&a
->	구조체 포인터 멤버 접근	p->x
.	구조체 멤버 접근	s.x
sizeof	크기(byte) 반환	sizeof(int)

마지막으로, 몇 가지 기타 기호들이 있는데, 이것들은 여러 역할로 자주 사용되거나(예: ,), 또는 여러 이유로 이전의 어떤 범주에도 잘 들어맞지 않는 것들이다.

연산자	의미	예시
,	순차 평가	a=1, b=2
()	함수 호출 / 우선순위	func()
[]	배열 접근	arr[i]
{ }	블록	{ int a; }

책을 계속 진행하면서 이 플래시 카드들을 꾸준히 공부하라. 만약 공부를 시작하기 전에 하루에 15~30 분, 그리고 잠자리에 들기 전에 또 15~30 분을 투자한다면, 아마 몇 주 안에 이 모든 것들을 외울 수 있을 것이다.

Exercise 6. C 문법 암기하기

연산자를 배운 후에는, 이제 여러분이 사용할 키워드와 기본 문법 구조를 암기할 차례이다. 이 작은 암기 시간은 책을 계속 진행해 나가면서 나중에 엄청난 보상으로 돌아올 것이라고 내가 말하는 것을 믿어도 된다.

Exercise 5에서 말했듯이, 이것들을 암기한다고 해서 책 읽기를 멈출 필요는 없다. 둘 다 동시에 할 수 있고, 또 그렇게 해야 한다. 매일 코딩을 사용하기 전에 플래시카드를 워밍업으로 사용하라. 플래시 카드를 꺼내서 15~30 분 정도 반복해서 외운 뒤, 자리에 앉아 책의 연습문제를 더 풀어라. 책을 진행하면서, 타이핑하는 코드를 암기한 내용을 연습하는 수단으로 활용하려고 노력하라. 한 가지 요령은, 코딩 중에 바로 떠오르지 않는 연산자나 키워드를 모아 별도의 플래시카드 더미를 만드는 것이다. 하루를 마친 뒤에는 그 카드들을 다시 15~30 분 동안 연습하라.

이 과정을 계속 유지하면, 그냥 코드를 두서없이 입력하면서 간접적으로 외우는 것보다 훨씬 빠르고 탄탄하게 C를 배우게 될 것이다.

[6-1] The Keywords(*플래시카드)

언어의 키워드는 기호(symbol)를 보완하여 언어가 더 읽기 쉬워지도록 하는 단어들이다. APL 처럼 키워드가 거의 없는 언어도 있고, Forth 나 LISP 처럼 거의 키워드로만 이루어진 언어도 있다. 그 중간에는 C, Python, Ruby 같은 언어들이 있으며, 이들은 키워드와 기호를 섞어서 언어의 기본을 만든다.

경고:

프로그래밍 언어의 기호와 키워드를 처리하는 기술적인 용어는 어휘 분석(lexical analysis)이다. 이러한 기호나 키워드 하나하나를 렉심(lexeme)이라고 부른다.

키워드	설명
auto	지역 변수에 지역 생명주기를 부여한다.
break	복합문(블록)에서 빠져나온다.
case	switch 문에서의 분기이다.
char	문자 데이터 타입이다.
const	변수를 수정 불가능하게 만든다.
continue	반복문의 맨 위로 이동한다.
default	switch 문에서의 기본 분기이다.
do	do-while 반복문을 시작한다.
double	배정밀도 부동소수점 데이터 타입이다.
else	if 문에서의 else 분기이다.
enum	정수 상수들의 집합을 정의한다.
extern	식별자가 외부에서 정의되었음을 선언한다.

float	부동소수점 데이터 타입이다.
for	for 반복문을 시작한다.
goto	레이블로 점프한다.
if	if 문을 시작한다.
int	정수 데이터 타입이다.
long	긴 정수 데이터 타입이다.
register	변수를 CPU 레지스터에 저장하도록 선언한다.
return	함수에서 반환한다.
short	짧은 정수 데이터 타입이다.
signed	정수 데이터 타입에 대한 signed 지정자이다.
sizeof	데이터의 크기를 결정한다.
static	스코프가 종료된 후에도 변수 값을 유지한다.
struct	변수들을 하나의 단일 레코드로 결합한다.
switch	switch 문을 시작한다.
typedef	새로운 타입을 생성한다.
union	union 문을 시작한다.
unsigned	정수 데이터 타입에 대한 unsigned 지정자이다.
void	비어 있는 데이터 타입을 선언한다.
volatile	변수가 다른 곳에서 변경될 수 있음을 선언한다.
while	while 반복문을 시작한다.

[6-2] Syntax Structures(문법 구조)

나는 키워드뿐만 아니라 문법 구조도 함께 암기할 것을 권한다. 문법 구조란 if문이나 while 반복문처럼 C 프로그램의 코드 형태를 이루는 기호들의 패턴이다. 여러분은 이미 하나 이상의 언어를 알고 있으므로 대부분 익숙할 것이다. 문제는 C가 그것을 어떻게 표현하는지를 배우는 것이다.

아래 문법을 읽는 방법은 다음과 같다.

1. 모두 대문자(ALLCAPS)로 된 부분은 교체될 자리, 즉 빈칸이다.
2. [ALLCAPS]처럼 대괄호로 감싸진 것은 선택 사항이다.
3. 문법 구조를 암기하는 가장 좋은 방법은 텍스트 편집기를 열고, 예를 들어, “switch 문”이라고 적힌 부분을 보고 그것이 무엇을 하는지 말한 뒤 코드 형태를 직접 써보는 것이다.

[6 -2-1] if 문

if 문은 기본적인 논리 분기 제어문이다.

```
if(TEST) {  
    CODE;  
} else if(TEST) {  
    CODE;  
} else {  
    CODE;  
}
```

[6 -2-2] switch 문

switch 문은 if 문과 비슷하지만, 단순한 정수 상수에 대해 동작한다.

```
switch (OPERAND) {  
case CONSTANT:  
    CODE;  
    break;  
default:  
    CODE;  
}
```

[6 -2-3] while 반복문

while 반복문은 가장 기본적인 반복문이다.

```
while(TEST) {  
    CODE;  
}
```

continue 를 사용해 반복을 다시 시작할 수도 있다. 이를 임시로 while-continue-loop 라고 부르자.

```
while(TEST) {  
    if(OTHER TEST) {  
        continue;  
    }  
    CODE;  
}
```

break 를 사용해 반복문을 종료할 수도 있다. 이를 while-break-loop 라고 부른다.

```
while(TEST) {  
    if(OTHER TEST) {  
        break;  
    }  
    CODE;  
}
```

[6 -2-4] do-while 반복문

do-while 반복문은 먼저 코드를 실행한 뒤, 다시 실행할지 여부를 검사하는 while 문의 반대 형태이다.

```
do {  
    CODE;  
} while(TEST);
```

이 안에서도 continue 와 break 를 사용해 동작을 제어할 수 있다.

[6 -2-5] for 반복문

for 반복문은 카운터를 사용하여 (보통) 고정된 횟수만큼 반복하는 제어된 반복문이다.

```
for(INIT; TEST; POST) {  
    CODE;  
}
```

[6 -2-6] enum

enum 은 정수 상수들의 집합을 만든다.

```
enum { CONST1, CONST2, CONST3 } NAME;
```

[6 -2-7] goto

goto 는 레이블로 점프하며, 오류 처리나 탈출 같은 몇 가지 유용한 상황에서만 사용된다.

```
if(ERROR TEST) {  
    goto fail;  
}  
fail:  
CODE;
```

[6 -2-8] 함수 정의

함수는 다음과 같이 정의한다.

```
TYPE NAME(ARG1, ARG2, ..) {  
    CODE;  
    return VALUE;  
}
```

이것이 어렵게 느껴질 수 있으므로, TYPE, NAME, ARG, VALUE 가 무엇인지 보여주는 예시는 다음과 같다.

```
int name(arg1, arg2) {  
    CODE;  
    return 0;  
}
```

[6 -2-9] typedef

typedef 는 새로운 타입을 정의한다.

```
typedef DEFINITION IDENTIFIER;
```

보다 구체적인 예시는 다음과 같다.

```
typedef unsigned char byte;
```

공백에 속지 말아라. 이 예제에서 DEFINITION 은 unsigned char 이고, IDENTIFIER 는 byte 이다.

[6 -2-10] struct

struct 는 여러 기본 데이터 타입을 하나의 개념으로 묶는 것으로, C 에서 매우 많이 사용된다.

```
struct NAME {  
    ELEMENTS;  
} [VARIABLE NAME];
```

[VARIABLE NAME]는 선택 사항이며, 나는 일부 작은 경우를 제외하고는 사용하지 않는 것을 선호한다. 이는 보통 typedef 와 함께 사용된다.

```
typedef struct [STRUCT NAME] {  
    ELEMENTS;  
} IDENTIFIER;
```


[6-2-11] union

마지막으로 union 은 struct 와 비슷하지만, 요솜르이 메모리 상에서 서로 겹친다. 이는 이해하기 다소 이상할 수 있으므로, 지금은 형태만 외워두자.

```
union NAME {  
    ELEMENTS;  
} [VARIABLE NAME];
```

[6-3] 격려의 말

이 각각에 대해 플래시카드를 만들고 나면, 보통의 방식대로 반복 학습을 하세요. 먼저 카드의 이름이 적힌 면부터 보고, 그다음 반대쪽에 있는 설명과 형태를 읽는 방식이다. 이 연습을 위한 영상에서 저는 Anki 를 사용해 이를 효율적으로 하는 방법을 보여주지만, 단순한 인덱스 카드로도 동일한 경험을 충분히 재현할 수 있다.

이런 내용을 암기하라는 요구를 받을 때, 일부 학생들에게서 두려움이나 느껴지는 것을 종종 본다. 왜 그런지는 정확히 모르겠지만, 그럼에도 불구하고 저는 여러분이 이 작업을 해보길르 권한다. 이것을 암기력과 학습 능력을 향상시킬 수 있는 기회라고 생각해 보자. 많이 할수록 점점 더 잘하게 되고, 점점 더 쉬워진다.

불편함과 좌절감을 느끼는 것은 아주 정상적인 일이다. 그것을 개인적인 문제로 받아들이지 말자. 15 분 정도를 보내면서 그저 이 일이 너무 싫고, 스스로를 완전한 실패자처럼 느낄 수도 있다. 이것은 정상이며, 실제로 여러분이 실패자라는 뜻은 아니다. 끈기는 초기의 좌절을 극복하게 해줄 것이고, 이 작은 연습은 여러분에게 두 가지를 가르쳐 줄 것이다.

첫째, 암기를 통해 자신의 역량을 스스로 평가할 수 있다는 점이다. 개념을 얼마나 잘 알고 있는지를 보여주는데에는 기억 테스트만큼 확실한 방법은 없다.

둘째, 어려움을 극복하는 방법은 한 번에 조금씩 해결하는 것이라는 점이다. 프로그래밍은 이를 배우기에 아주 좋은 분야인데, 문제를 아주 작은 단위로 나누고 부족한 부분에 집중하기가 매우 쉽게 때문이다. 이 과정을 큰 작업을 작은 조각으로 나누어 해결하는 자신감을 기르는 기회로 삼자.

[6-4] 주의의 말

마지막으로 암기에 대해 한 가지 경고의 말을 덧붙이겠다. 많은 양의 사실을 암기한다고 해서 자동으로 그것을 잘 활용할 수 있게 되는 것은 아니다. ANSI C 표준 문서를 통째로 외우고도 형편없는 프로그래머일 수 있다. 저는 표준 C 문법의 모든 구석구석을 알고 있으면서도 여전히 형편없고, 버그가 많고, 이상한 코드를 작성하거나 아예 코딩을 하지 않는 소위 C 전문가들을 많이 만났다.

암기한 사실을 그대로 읊을 수 있는 능력과 실제로 무언가를 잘 해낼수 있는 능력을 절대 혼동하지 말자. 그것을 제대로 하기 위해서는, 이 사실들을 다양한 상황에 적용해 보면서 실제로 어떻게 사용하는지 익혀야 한다. 이 책의 나머지 부분이 바로 그것을 도와줄 것이다.

Exercise 7. 변수와 타입

이제 간단한 C 프로그램의 구조가 어떻게 되어 있는지를 감을 잡았을 것이므로, 다음으로 가장 단순한 일을 해보자. 바로 서로 다른 타입의 변수들을 만들어 보는 것이다.

[7-1] ex7.c

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    int distance = 100;
    float power = 2.345f;
    double super_power = 56789.4532;
    char initial = 'A';
    char first_name[] = "Zed";
    char last_name[] = "Shaw";

    printf("You are %d miles away.\n", distance);
    printf("You have %f levels of power.\n", power);
    printf("You have %f awesome super powers.\n", super_power);
    printf("I have an initial %c.\n", initial);
    printf("I have a first name %s.\n", first_name);
    printf("I have a last name %s.\n", last_name);
    printf("My whole name is %s %c. %s.\n",
           first_name, initial, last_name);

    int bugs = 100;
    double bug_rate = 1.2;

    printf("You have %d bugs at the imaginary rate of %f.\n",
           bugs, bug_rate);

    long universe_of_defects = 1L * 1024L * 1024L * 1024L;

    printf("The entire universe has %ld bugs.\n",
           universe_of_defects);

    double expected_bugs = bugs * bug_rate;

    printf("You are expected to have %f bugs.\n",
           expected_bugs);
```

```

double part_of_universe = expected_bugs /
    universe_of_defects;

printf("That is only a %e portion of the universe.\n",
    part_of_universe);

// this makes no sense, just a demo of something weird
char nul_byte = '\0';
int care_percentage = bugs * nul_byte;

printf("Which means you should care %d%%.\n",
    care_percentage);

return 0;
}

```

이 소스 파일은 서로 다른 타입의 변수들이 있을 때 수학 연산이 어떻게 동작하는지를 보여준다. 또한 프로그램의 마지막 부분에서는 C에서는 볼 수 있지만 다른 많은 언어에서는 보기 힘든 한 가지를 보여준다.

C에서 문자는 단순히 정수일 뿐이다. 아주 작은 정수이긴 하지만, 결국 정수다. 이 말은 문자에도 수학연산을 할 수 있다는 뜻이며, 실제로 많은 소프트웨어가 좋은 나쁜든 그렇게 하고 있다.

이 마지막 부분은 C가 어떻게 여러분에게 기계에 대한 직접적인 접근을 허용하는지를 처음으로 엿보게 해준다. 이에 대해서는 이후 연습 문제들에서 더 자세히 살펴볼 것이다.

[7-2] 실행 결과

평소처럼, 실행 결과는 다음과 같아야 한다.

```

$ make ex7
cc -Wall -g ex7.c -o ex7
$ ./ex7
You have 100 bugs at the imaginary rate of 1.200000.
The entire universe has 1073741824 bugs.
You are expected to have 120.000000 bugs.
That is only a 1.117587e-07 portion of the universe.
Which means you should care 0%.
$

```

[7-3] 망가뜨려 보기(How to Break It)

다시 한 번 말하지만, 이 코드를 살펴보고 printf 에 잘못된 인자를 넘겨서 깨뜨려 보자.
nul_byte 변수를 %s 로 출력하려고 할 때와 %c 로 출력하려고 할 때 어떤 일이 발생하는지도 확인해 보라.

프로그램이 깨졌다면, 디버거에서 실행해 보고 네가 무엇을 잘못했는지 디버거가 뭐라고 말하는지도 확인해 보자.

[7-4] 추가 과제(Extra Credit)

- universe_of_defects 에 할당하는 숫자의 크기를 여러 가지로 바꿔서 컴파일러 경고가 발생할 때까지 시도해 보자.
- 이런 정말로 큰 숫자들은 실제 어떻게 출력되는가?
- long 을 unsigned long 으로 바꾸고, 너무 커지는 숫자를 찾아봐라.
- unsigned 가 무엇인지 온라인에서 검색하자.
- 왜 char 와 int 를 곱할 수 있는지에 대해 (다음 연습문제에서 설명하기 전에) 스스로 설명해 보자.

Exercise 8. if, else-if, else

C 언어에는 실제로 Boolean 타입이 존재하지 않는다. 대신, 정수형 값에서 0 은 거짓(false)으로 간주되고, 0 이 아닌 모든 값은 참(true)로 간주된다.

이전 Exercise 에서 `argc > 1` 이라는 표현식은 실제로 1 또는 0 을 반환했지, Python 처럼 명시적인 True 나 False 를 반환하지 않는다. 이것은 C 가 컴퓨터가 실제로 동작하는 방식에 더 가깝다는 또 다른 예이다. 컴퓨터에게 참/거짓 값은 결국 정수이기 때문이다.

하지만 C 에는 이런 숫자 기반 참/거짓을 이용한 if문이 있다. 동작 방식은 Python 이나 Ruby 와 비슷하다. 이번 연습문제를 보면 이해가 쉽다.

[8-1] ex8.c

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    int i = 0;

    if (argc == 1) {
        printf("You only have one argument. You suck.\n");
    } else if (argc > 1 && argc < 4) {
        printf("Here's your arguments:\n");

        for (i = 0; i < argc; i++) {
            printf("%s ", argv[i]);
        }
        printf("\n");
    } else {
        printf("You have too many arguments. You suck.\n");
    }

    return 0;
}
```

if 문의 형식은 다음과 같다.

```
if (TEST) {  
    CODE;  
} else if (TEST) {  
    CODE;  
} else {  
    CODE;  
}
```

대부분의 언어와 비슷하지만, C에는 몇 가지 특징이 있다.

- 앞서 말했듯이, TEST 부분이 0이면 거짓, 0이 아니면 참으로 간주된다.
- TEST 부분은 반드시 괄호 ()로 감싸야 한다. 다른 언어에서는 생략 가능한 경우가 있다.
- 코드 블록을 {}로 감싸는 것이 필수는 아니지만, 생략하면 안 좋은 습관이다. 괄호를 사용하면 각 부분(branch)의 시작과 끝이 명확해진다. 생략하면 실수하기 쉽다.

그 외에는 다른 언어와 거의 동일하게 작동한다. else if 나 else 를 반드시 사용할 필요는 없다.

[8-2] 실행 결과 예시

아주 간단하므로 직접 실행해보면 쉽게 이해할 수 있다.

```
$ make ex8  
cc -Wall -g ex8.c -o ex8  
  
$ ./ex8  
You only have one argument. You suck.  
  
$ ./ex8 one  
Here's your arguments:  
./ex8 one  
  
$ ./ex8 one two  
Here's your arguments:  
./ex8 one two  
  
$ ./ex8 one two three  
You have too many arguments. You suck.
```

[8-3] 망가뜨려 보기(How to Break It)

이 프로그램은 매우 간단해서 쉽게 깨지지 않지만, if 문의 조건을 바꾸면 테스트할 수 있다.

- 마지막 else 를 제거하면 예외 케이스를 잡지 못하게 된다.
- &&를 ||로 바꾸면, and 조건 대신 or 조건이 되어 결과가 달라진다.

[8-4] 추가 과제(Extra Credit)

- 앞서 잠깐 소개된 &&는 and 연산자이다. 온라인에서 다른 Boolean 연산자(||, !)도 조사해보자.
- 프로그램에 대해 몇 가지 더 테스트 케이스를 만들자
- 첫 번째 테스트가 정말 올바르게 동작하는지 생각해보자. 사용자가 입력한 첫 번째 인자가 실제 프로그램에서 의미하는 argv[0]과 다를 수 있다. 필요하다면 수정하자.

Exercise 9. while 루프와 Boolean 표현식

이번에 소개할 반복문 구조는 while 루프이다. C에서 사용할 수 있는 가장 단순하고 유용한 반복문 중 하나이다. 이번 연습문제 코드가 아래와 같다.

[9-1] ex9.c

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    int i = 0;
    while (i < 25) {
        printf("%d", i);
        i++;
    }
    return 0;
}
```

이 코드와 기본 문법을 보면, while 루프의 구조는 이렇게 단순하다.

```
while (TEST) {
    CODE;
}
```

대부분의 언어와 비슷하지만, C에는 몇 가지 특징이 있다.

- TEST가 참(1)인 동안 CODE를 반복 실행한다.
- for 루프와 동일한 동작을 만들고 싶다면, i의 초기화와 증가를 직접 처리해야 한다.
- i++는 후위 증가 연산자(post-increment operator)로, i를 1증가시킨다.
 - 만약 이 연산자를 몰랐다면, 이전에 배운 토큰(token)리스트를 다시 확인하자.

[9-2] 실행 결과 예시

while 루프는 기본적으로 for 루프와 비슷한 출력을 내지만, 조금 다른 방식으로 실행된다.

```
$ make ex9
cc -Wall -g ex9.c -o ex9

$ ./ex9
arg 0: ./ex9
state 0: California
state 1: Oregon
state 2: Washington
```

```
state 3: Texas
```

```
$ ./ex9 test it
```

```
arg 0: ./ex9
```

```
arg 1: test
```

```
arg 2: it
```

```
state 0: California
```

```
state 1: Oregon
```

```
state 2: Washington
```

```
state 3: Texas
```

[9-3] 망가뜨려 보기(How to Break It)

while 루프는 실수로 잘못 작성하기 쉽다. 실제로 초보자가 많이 겪는 문제들이 있다.

- 첫 번째 i 초기화를 깜빡함
 - i가 어떤 값으로 시작하느냐에 따라 루프가 전혀 실행되지 않거나, 매우 오래 실행될 수 있다.
- 두 번째 루프의 i 초기화 깜빡함
 - 이전 루프의 i 값을 그대로 쓰게 되어, 두 번째 루프가 실행될 수도, 안될 수도 있음
- i++를 루프 끝에서 생략함
 - 무한루프가 발생한다. 프로그래밍 초반에 가장 흔하게 발생하는 문제 중 하나이다.

[9-4] 추가 과제(Extra Credit)

- i를 25에서 0으로 감소시키는 루프를 만들어 보자(i--사용)
- 지금까지 배운 지식을 활용해 더 복잡한 while 루프를 여러 개 만들어 보자

Exercise 10. Switch 문

다른 언어들, 예를 들어 Ruby에서는 switch 문이 어떤 표현식이든 받을 수 있다.

Python 같은 언어에는 switch 문이 없는데, 그 이유는 if 문과 Boolean 표현식만으로 동일한 기능을 구현할 수 있기 때문이다.

즉, 이런 언어에서는 switch 문은 단순히 if 문의 대안일 뿐이고, 내부 동작도 거의 동일하다.

하지만 C에서의 switch 문은 조금 다르다. 실제로 점프 테이블(jump table)로 동작한다.

- 일반 Boolean 표현식을 넣을 수 없고, 정수값으로 평가되는 표현식만 넣을 수 있다.
- 이 정수값을 기반으로 switch 문의 시작 지점에서 해당 값에 맞는 위치로 점프한다.

[10-1] ex10.c

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    int i = 0;

    // argv 의 각 문자열을 순회
    // argv[0]는 왜 건너뛰을까?
    for (i = 1; i < argc; i++) {
        printf("arg %d: %s\n", i, argv[i]);
    }

    // 문자열 배열을 만들어보자
    char *states[] = {
        "California",
        "Washington",
        "Oregon",
        "Texas",
    };

    int num_states = 4;

    for (i = 0; i < num_states; i++) {
        printf("state %d: %s\n", i, states[i]);
    }

    return 0;
}
```

[10-2] switch 문의 동작 원리

1. 컴파일러는 프로그램에서 switch 문이 시작되는 위치를 표시한다. 이를 Y 라고 한다.
2. switch(letter) 안의 표현식을 평가하여 정수값을 계산한다.
 - 이 예제에서는 argv[1]의 글자 ASCII 코드가 된다.
3. 각 case 블록(case 'A:')을 프로그램 내의 위치로 변환한다.
 - 예: case 'A' 아래 코드는 Y + 'A'이다.
4. 컴파일러는 Y+letter 위치를 계산하고, 범위를 벗어나면 Y + default 로 이동한다.
5. 위치가 결정되면 프로그램은 해당 코드로 점프해 실행을 계속한다.
 - 이 때문에 일부 case 블록에는 break 가 필요하다.
6. 예를 들어, 'a'가 입력되면 case 'a'로 점프하고, 그 아래 코드를 “fall through”하여 다음 case 까지 실행될 수 있다.
7. 코드가 실행되면 break 를 만나 switch 문을 완전히 종료한다.

[10-3] 실무에서 기억할 점

- 항상 default 분기를 포함하자
 - 누락된 입력을 처리할 수 있다.
- 불필요한 fall-through 는 피하자.
 - 의도적으로 사용하는 경우 //fall through 주석을 추가
- case 와 break 를 코드 작성 전에 먼저 작성하자.
- 가능하면 if 문으로 대체하는 것이 더 안전할 수 있다.

[10-4] 실행 결과 예시

```
$ make ex10
cc -Wall -g ex10.c -o ex10

$ ./ex10
ERROR: You need one argument.

$ ./ex10 Zed
0: Z is not a vowel
1: 'E'
2: d is not a vowel

$ ./ex10 Zed Shaw
0: Z is not a vowel
1: 'E'
2: d is not a vowel
3: is not a vowel
4: S is not a vowel
```

5: h is not a vowel

6: 'A'

7: w is not a vowel

[10-5] 망가뜨려 보기(How to Break It)

- break 를 잊으면 원하지 않으나 여러 블록이 실행된다.
- default 를 잊으면 누락된 값이 무시된다.
- switch 안에 예상치 못한 값(예: int 변수)을 넣으면 이상한 동작이 발생한다.
- 초기화되지 않은 값을 사용하면 프로그램이 예기치 않게 동작할 수 있다.

[10-6] 추가 과제(Extra Credit)

- 글자를 소문자로 변환하는 수학 연산을 적용해 switch 문에서 대문제를 제거하자.
- for 문에서 ,(comma)를 사용하여 letter 를 초기화하자
- 모든 인자를 처리하도록 또 다른 for 문을 추가하자
- 이 switch 문을 if 문으로 바꾸자. 어느 방식이 더 좋은지 비교하자.
- 'Y' case 에서 break 를 if 문 안에 넣으면 어떤 영향이 있는지 실험해보자.

Exercise 10. Switch 문

다른 언어들, 예를 들어 Ruby에서는 switch 문이 어떤 표현식이든 받을 수 있다.

Python 같은 언어에는 switch 문이 없는데, 그 이유는 if 문과 Boolean 표현식만으로 동일한 기능을 구현할 수 있기 때문이다.

즉, 이런 언어에서는 switch 문은 단순히 if 문의 대안일 뿐이고, 내부 동작도 거의 동일하다.

하지만 C에서의 switch 문은 조금 다르다. 실제로 점프 테이블(jump table)로 동작한다.

- 일반 Boolean 표현식을 넣을 수 없고, 정수값으로 평가되는 표현식만 넣을 수 있다.
- 이 정수값을 기반으로 switch 문의 시작 지점에서 해당 값에 맞는 위치로 점프한다.

[10-1] ex10.c

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    int i = 0;

    // argv 의 각 문자열을 순회
    // argv[0]는 왜 건너뛰을까?
    for (i = 1; i < argc; i++) {
        printf("arg %d: %s\n", i, argv[i]);
    }

    // 문자열 배열을 만들어보자
    char *states[] = {
        "California",
        "Washington",
        "Oregon",
        "Texas",
    };

    int num_states = 4;

    for (i = 0; i < num_states; i++) {
        printf("state %d: %s\n", i, states[i]);
    }

    return 0;
}
```

[10-2] switch 문의 동작 원리

1. 컴파일러는 프로그램에서 switch 문이 시작되는 위치를 표시한다. 이를 Y 라고 한다.
2. switch(letter) 안의 표현식을 평가하여 정수값을 계산한다.
 - 이 예제에서는 argv[1]의 글자 ASCII 코드가 된다.
3. 각 case 블록(case 'A:')을 프로그램 내의 위치로 변환한다.
 - 예: case 'A' 아래 코드는 Y + 'A'이다.
4. 컴파일러는 Y+letter 위치를 계산하고, 범위를 벗어나면 Y + default 로 이동한다.
5. 위치가 결정되면 프로그램은 해당 코드로 점프해 실행을 계속한다.
 - 이 때문에 일부 case 블록에는 break 가 필요하다.
6. 예를 들어, 'a'가 입력되면 case 'a'로 점프하고, 그 아래 코드를 “fall through”하여 다음 case 까지 실행될 수 있다.
7. 코드가 실행되면 break 를 만나 switch 문을 완전히 종료한다.

[10-3] 실무에서 기억할 점

- 항상 default 분기를 포함하자
 - 누락된 입력을 처리할 수 있다.
- 불필요한 fall-through 는 피하자.
 - 의도적으로 사용하는 경우 //fall through 주석을 추가
- case 와 break 를 코드 작성 전에 먼저 작성하자.
- 가능하면 if 문으로 대체하는 것이 더 안전할 수 있다.

[10-4] 실행 결과 예시

```
$ make ex10
cc -Wall -g ex10.c -o ex10

$ ./ex10
ERROR: You need one argument.

$ ./ex10 Zed
0: Z is not a vowel
1: 'E'
2: d is not a vowel

$ ./ex10 Zed Shaw
0: Z is not a vowel
1: 'E'
2: d is not a vowel
3: is not a vowel
4: S is not a vowel
```

5: h is not a vowel
6: 'A'
7: w is not a vowel

[10-5] 망가뜨려 보기(How to Break It)

- break 를 잊으면 원하지 않으나 여러 블록이 실행된다.
- default 를 잊으면 누락된 값이 무시된다.
- switch 안에 예상치 못한 값(예: int 변수)을 넣으면 이상한 동작이 발생한다.
- 초기화되지 않은 값을 사용하면 프로그램이 예기치 않게 동작할 수 있다.

[10-6] 추가 과제(Extra Credit)

- 글자를 소문자로 변환하는 수학 연산을 적용해 switch 문에서 대문제를 제거하자.
- for 문에서 ,(comma)를 사용하여 letter 를 초기화하자
- 모든 인자를 처리하도록 또 다른 for 문을 추가하자
- 이 switch 문을 if 문으로 바꾸자. 어느 방식이 더 좋은지 비교하자.
- 'Y' case 에서 break 를 if 문 안에 넣으면 어떤 영향이 있는지 실험해보자.

Exercise 11. 배열과 문자열

이번 연습문제에서는 C 에서 문자열이 실제로 어떻게 저장되는지 보여준다.

C 는 문자열을 단순히 바이트 배열(array of bytes)로 저장하며, 마지막은 `\0`(널 문자)로 끝난다.

- 이전 연습문제에서 이미 수동으로 배열을 다뤄보면서 눈치 챘을 것이다.
- 이번에는 숫자 배열과 비교하여 더 명확하게 보여준다.

[11-1] ex11.c

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    int numbers[4] = {0};
    char name[4] = {'a'};

    // 처음에 배열 출력
    printf("numbers: %d %d %d %d\n",
           numbers[0], numbers[1], numbers[2], numbers[3]);

    printf("name each: %c %c %c %c\n",
           name[0], name[1], name[2], name[3]);

    printf("name: %s\n", name);

    // 숫자 배열 초기화
    numbers[0] = 1;
    numbers[1] = 2;
    numbers[2] = 3;
    numbers[3] = 4;

    // 문자열 초기화
    name[0] = 'Z';
    name[1] = 'e';
    name[2] = 'd';
    name[3] = '\0';

    // 초기화 후 출력
    printf("numbers: %d %d %d %d\n",
           numbers[0], numbers[1], numbers[2], numbers[3]);
```

```

printf("name each: %c %c %c %c\n",
      name[0], name[1], name[2], name[3]);

printf("name: %s\n", name);

// 문자열 또 다른 방법
char *another = "Zed";

printf("another: %s\n", another);

printf("another each: %c %c %c %c\n",
      another[0], another[1], another[2], another[3]);

return 0;
}

```

[11-2] 핵심 포인트

1. 배열 초기화

- int numbers[4] = {0}; 처럼 한 요소만 초기화하면 나머지는 0 으로 채워진다.
- char name[4] = {'a'};도 첫 글자만 지정하면 나머지는 자동으로 \0 이 들어간다.

2. 문자 배열 출력

- 각 요소를 %c 로 출력하면, \0 이후 문자는 보이지 않는다.
- %s 로 출력하면 문자열처럼 출력된다.
- 배열과 문자열의 차이가 사실상 없다. C 에서 문자열은 문자 배열과 동일하게 취급된다.

3. 문자열 초기화 방식

- char name[4] = {'a', 'b', ...}; : 수동 초기화, 덜 일반적
- char *another = "Zed"; : 문자열 리터럴, 더 일반적이고 직관적

4. 숫자 배열과 문자열 배열 비교

- numbers 배열과 name 배열을 같은 방식으로 다뤄도, printf 에서 문자열로 인식하면 %s 로 출력된다.

[11-3] 실행 결과 예시

```
$ make ex11
cc -Wall -g ex11.c -o ex11
$ ./ex11
numbers: 0 0 0 0
name each: a
name: a
numbers: 1 2 3 4
name each: Z e d
name: Zed
another: Zed
another each: Z e d
```

- 처음에는 숫자와 문자 배열이 모두 초기값 0 또는 'a'로 출력된다.
- 수동으로 값을 넣으면 숫자는 바뀌고, 문자열도 %s 로 올바르게 출력된다.

[11-4] 망가뜨려 보기(How to Break It)

C 에서 거의 모든 버그는 공간 부족 또는 문자열 끝에 \0 을 넣지 않은 것에서 발생한다.

- 초기화하지 않고 출력
- 문자열 마지막에 \0 대신 다른 문자를 넣음
- 배열 전체를 'a', 'a', 'a', 'a'로 채워 \0 공간이 없음

이런 문제는 디버거로도 잡기 어려운 경우가 있다.

- 변수 위치를 바꾸기만 해도 버그가 발생할 수 있다.
- C 의 특성상 배치에 따라 버그가 달라지는 경우가 많다.

[11-5] 추가 과제(Extra Credit)

- 숫자 배열에 문자를 넣고 한 글자씩 출력, 컴파일러 경고 확인
- 문자열 배열을 숫자 배열처럼 다뤄보기
- 다양한 방법으로 배열 출력
- 4 바이트 문자 배열과 4 바이트 정수 배열을 같이 취급하는 해킹 실험
- 배열을 그림으로 그리고 연산을 손으로 따라하기
- name 을 char *another 스타일로 바꿔도 코드가 계속 동작하는 지 확인

Exercise 12. 배열과 크기

이전 연습문제에서는 ‘W0’(널 문자)을 사용한 수학적 연산을 해봤다.

다른 언어에서는 문자열과 바이트 배열을 구분하려고 하지만, C에서는 문자열도 단순히 바이트 배열이다.

단지 출력함수(printf 등)가 차이를 인식할 뿐이다.

이번에는 배열과 sizeof 개념을 함께 배운다.

[12-1] ex12.c

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    int areas[] = {10, 12, 13, 14, 20};
    char name[] = "Zed";
    char full_name[] = {
        'Z','e','d',' ',
        'A','.',',',
        'S','h','a','w',
        'W0'
    };

    // int 관련 크기
    printf("The size of an int: %ldWn", sizeof(int));
    printf("The size of areas (int[]): %ldWn", sizeof(areas));
    printf("The number of ints in areas: %ldWn", sizeof(areas)/sizeof(int));
    printf("The first area is %d, the 2nd %d.Wn", areas[0], areas[1]);

    // char 관련 크기
    printf("The size of a char: %ldWn", sizeof(char));
    printf("The size of name (char[]): %ldWn", sizeof(name));
    printf("The number of chars: %ldWn", sizeof(name)/sizeof(char));

    printf("The size of full_name (char[]): %ldWn", sizeof(full_name));
    printf("The number of chars: %ldWn", sizeof(full_name)/sizeof(char));

    printf("name=W"%sW" and full_name=W"%sW"Wn", name, full_name);

    return 0;
}
```

[12-2] 핵심 포인트

1. 배열 생성 방식

- `type name[] = {initializer};` 구문으로 배열을 초기화한다.
- 예: `int areas[] = {10, 12, 13, 14, 20};`
 - int 형 5 개를 담을 배열을 만들고, 이름은 areas 이다.
- `char name[] = "Zed";`
 - 문자열을 문자 배열로 초기화한다.
- `full_name[] = {...}`
 - 한 글자씩 명시적으로 초기화한 문자 배열이다.
- C 에서는 이 두 방법이 동일하게 char 배열을 생성한다.

2. sizeof 연산자

- `sizeof(변수)`로 변수나 배열의 메모리 크기(바이트 단위)를 알 수 있다.
- int 배열이면 배열 전체 크기/int 크기 = 요소 개수로 계산할 수 있다.
- char 배열도 마찬가지이다.

[12-3] 실행 결과 예시

```
$ ./ex12
The size of an int: 4
The size of areas (int[]): 20
The number of ints in areas: 5
The first area is 10, the 2nd 12.
The size of a char: 1
The size of name (char[]): 4
The number of chars: 4
The size of full_name (char[]): 12
The number of chars: 12
name="Zed" and full_name="Zed A. Shaw"
```

- int 크기: 4 바이트(32 비트 기준)
- areas 배열: 5 개의 int: 20 바이트
- name 배열: "Zed" + `\0` : 4 바이트
- full_name 배열: 한 글자씩 명시: 12 바이트
- C 는 배열 길이를 자동으로 계산하고, 문자열 끝에 `\0` 을 추가한다.

[12-4] 망가뜨려 보기(How to Break It)

- full_name 마지막에 `\0` 을 빼먹으면 문자열 출력시 오류 발생
- 배열 범위를 벗어난 접근: `areas[10]`
- name 이나 full_name 에도 범위 초과나 초기화 안함
- 디버거로 확인하면 변수 위치나 초기화 순서에 따라 예상치 못한 버그가 생길 수 있음

[12-5] 추가 과제(Extra Credit)

- `areas[0] = 100;` 등 배열 요소 직접 변경
- `name` 이나 `full_name` 배열 요소 수정
- 배열에 다른 타입 할당 시도(문자를 `int` 로 넣기 등)
- CPU 별 `int` 크기 비교
- 배열을 그림으로 그려서 메모리 구조 이해
- 문자열을 `char *another = "Literal";` 스타일로 바꿔 코드가 계속 동작하는지 확인

Exercise 13. for 문과 문자열

C에서 문자열과 바이트 배열은 동일하게 취급할 수 있다.

이번 연습에서는 문자열을 요소로 갖는 배열을 만들고, for 문을 사용하여 출력해본다.

흥미로운 점은 이미 프로그램 안에 문자열 배열이 숨어 있었다는 것이다.

- char *argv[]가 바로 명령줄 인수를 담은 문자열 배열이다.

[13-1] ex13.c

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    if (argc != 2) {
        printf("ERROR: You need one argument.\n");
        return 1; // 프로그램 종료
    }

    int i = 0;
    for (i = 0; argv[1][i] != '\0'; i++) {
        char letter = argv[1][i];

        switch (letter) {
            case 'a': case 'A':
                printf("%d: 'A'\n", i);
                break;
            case 'e': case 'E':
                printf("%d: 'E'\n", i);
                break;
            case 'i': case 'I':
                printf("%d: 'I'\n", i);
                break;
            case 'o': case 'O':
                printf("%d: 'O'\n", i);
                break;
            case 'u': case 'U':
                printf("%d: 'U'\n", i);
                break;
            case 'y': case 'Y':
                if (i > 2) {
                    printf("%d: 'Y'\n", i);
                }
            }
        }
    }
}
```

```

        }
        break;
    default:
        printf("%d: %c is not a vowel\n", i, letter);
    }
}

return 0;
}

```

[13-2] for 문 구조

```

for (INITIALIZER; TEST; INCREMENTER) {
    CODE;
}

```

- INITIALIZER: 루프 변수 초기화(i = 0)
 - TEST: 반복 조건 검사(argv[1][i] != 'W0')
 - CODE: 루프 안에서 실행될 코드
 - INCREMENTER: 루프 변수 증가(i++)
- 루프 조건이 false(0)가 될 때까지 반복된다.

[13-3] 명령줄 인수 처리

1. OS 는 프로그램 실행 시 인수를 argv 배열에 문자열로 전달한다.
2. argc 는 argv 배열의 길이(인수 개수)를 나타낸다.
 - 프로그램 이름(./ex13)은 argv[0]
 - 첫 번째 인수는 argv[1]
3. 위 for 문은 argv[1] 문자열의 각 문자를 순회하며 모음을 판별한다.

[13-4] 실행 결과 예시

```

$ ./ex13 i
0: i

$ ./ex13 hello
0: h is not a vowel
1: 'E'
2: l is not a vowel
3: l is not a vowel
4: 'O'

```

- 인수를 주지 않으면, argc != 2 조건에 걸려 오류 출력 후 종료

[13-5] 문자열 배열 이해

- C 에서 문자열 배열(array of strings)은 2 차원 배열과 유사하다.
- 예:

```
char *states[] = {"California", "Oregon", "Washington", "Texas"};
```

- 각 문자열이 배열 요소, 문자열 안의 각 문자가 또 다른 배열 요소이다.
- 이 개념이 헷갈리면 종이에 격자로 그려서 인덱스 확인하면 이해가 쉽다.

[13-6] 망가뜨려 보기(How to Break It)

- 아주 많은 명령줄 인수 넣어보기
- i = 0 초기화 후 인덱스 문제 확인
- num_states 를 잘못 설정하고 실행
- 문자열 배열 범위를 넘어가거나 NULL 요소 사용

[13-7] 추가 과제(Extra Credit)

- for 문의 각 부분에 다양한 코드를 넣어보기
- ,(콤마) 연산자로 여러 문장 연결
- 문자 배열 요소에 NULL 넣고 출력 결과 확인
- 문자열 배열과 argv 배열 간 요소를 서로 할당해보기

Exercise 14. 함수 작성과 사용

지금까지 우리는 `stdio.h` 헤더 파일에 포함된 함수들만 사용해 왔다. 이번 장에서는 직접 함수를 작성하고, 다른 함수들도 사용한다.

[14-1] ex14.c

```
#include <stdio.h>
#include <ctype.h>

// forward declarations
int can_print_it(char ch);
void print_letters(char arg[]);
void print_arguments(int argc, char *argv[]);

void print_arguments(int argc, char *argv[])
{
    int i = 0;

    for (i = 0; i < argc; i++) {
        print_letters(argv[i]);
    }
}

void print_letters(char arg[])
{
    int i = 0;

    for (i = 0; arg[i] != '\0'; i++) {
        char ch = arg[i];

        if (can_print_it(ch)) {
            printf("%c' == %d ", ch, ch);
        }
    }

    printf("\n");
}

int can_print_it(char ch)
{
    return isalpha(ch) || isblank(ch);
}
```

```

}

int main(int argc, char *argv[])
{
    print_arguments(argc, argv);
    return 0;
}

```

[14-2] 설명

이 예제에서는 알파벳 문자이거나 공백인 문자들만 골라서,
그 문자와 ASCII 코드를 출력하는 함수들을 만들고 있다.

- 2 번째 줄

- 새로운 헤더 파일을 포함한다.
- isalpha 와 isblank 함수를 사용하기 위해 <ctype.h>를 포함한다.

- 5~6 번째 줄

앞으로 이 프로그램에서 사용할 함수들을 미리 선언한다.

이것을 forward declaration(전방 선언)이라고 한다.

이것은 아직 정의하지 않은 함수를 먼저 사용해야 하는 “닭이 먼저냐 달걀이 먼저냐” 문제를 해결해 준다.

- 8~15 번째 줄

print_arguments 함수를 정의한다.

이 함수는 main 이 일반적으로 받는 문자열 배열(argv)을 출력하는 방법을 알고 있다.

- 17~30 번째 줄

print_letters 함수를 정의한다.

이 함수는 isalpha(ch) || isblank(ch)의 결과(0 또는 1)를 반환한다.

즉, 문자가 알파벳이거나 공백이면 참(1),

아니면 거짓(0)을 반환한다.

- 37~41 번째 줄

마지막으로 main 함수는 print_arguments 를 호출한다.

이렇게 해서 함수 호출이 연쇄적으로 실행된다.

각 함수 내부의 코드는 이미 이전에 배운 내용이므로 하나하나 다시 설명하지 않는다.
여기서 중요한 점은 main 을 정의했던 것과 똑같은 방식으로 다른 함수들도 정의할 수 있다는 것이다. 단, 아직 정의되지 않은 함수를 먼저 사용할 경우 전방선언이 필요하다.

[14-3] 실행 결과 예시

이 프로그램을 실행하려면 다양한 명령줄 인자를 전달하면 된다.

```
$ make ex14
cc -Wall -g ex14.c -o ex14

$ ./ex14
'e' == 101 'x' == 120

$ ./ex14 hi this is cool
'e' == 101 'x' == 120
'h' == 104 'i' == 105
't' == 116 'h' == 104 'i' == 105 's' == 115
'i' == 105 's' == 115
'c' == 99 'o' == 111 'o' == 111 'l' == 108

$ ./ex14 "I go 3 spaces"
'e' == 101 'x' == 120
'I' == 73 ' ' == 32 'g' == 103 'o' == 111 ' ' == 32 ' ' == 32
's' == 115 'p' == 112 'a' == 97 'c' == 99 'e' == 101 's' == 115
```

마지막 실행에서 숫자 3 은 출력되지 않는다.

왜냐하면 3 은 알파벳도 아니고 공백도 아니기 때문이다.

[14-4] 망가뜨려 보기(How to Break It)

이 프로그램은 두 가지 방법으로 망가뜨릴 수 있다.

1. 전방 선언을 제거하기

can_print_it 과 print_letters 의 전방 선언을 지워보자. 컴파일러가 에러를 낼 것이다.

2. main 에서 argc 에 1 을 더해보기

```
print_arguments(argc + 1, argv);
이렇게 하면 argv 배열의 끝을 넘어가게 된다.
결과:
- 이상한 값 출력
- 프로그램 충돌
- 정의되지 않은 동작
```

[14-5] 추가 과제(Extra Credit)

- 함수 개수를 줄여보기
 - 예를 들면, `can_print_if` 이 정말 필요한가
- `print_arguments` 에서 `strlen` 함수를 사용해 각 문자열의 길이를 계산한 뒤 그 길이를 `print_letters` 에 전달하기. 그리고 `print_letters` 가 'W0'에 의존하지 않도록 수정하기
- `man` 명령으로 `isalpha`, `isblank` 를 찾아보기
 - `isdigit` 같은 다른 함수도 사용하기
- 다른 사람들의 함수 작성 스타일을 읽어보기. 단 K&R 문법은 사용하지 말자.

Exercise 15. 포인터, 두려운 포인터

포인터는 C 에서 유명한 신비로운 존재이다.

저는 여러분이 포인터를 다룰 수 있도록 어휘를 가르쳐서 그 신비를 풀어보려 한다.

사실 그렇게 복잡하지는 않지만, 사람들이 이상한 방식으로 남용하는 경우가 많아서 사용하기 어렵게 보인다. 어리석은 방식으로 포인터를 사용하지만 않으면, 꽤 간단하다. 포인터를 설명하기 위해, 사람들의 나이를 세 가지 방식으로 출력하는 프로그램을 작성한다.

[15-1] ex15.c

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    //우리가 사용할 두 개의 배열 생성
    int ages[] = {23, 43, 12, 89, 2};
    char *names[] = {
        "Alan", "Frank", "Mary", "John", "Lisa"
    };

    //ages 배열의 크기를 안전하게 구하기
    int count = sizeof(ages) / sizeof(int);
    int i = 0

    // 첫 번째 방법: 인덱싱 사용
    for(i = 0; i < count; i++){
        printf("%s has %d years alive.\n", names[i], ages[i]);
    }

    printf("\n--\n");

    //배열을 시작을 가리키는 포인터 설정
    int *cur_age = ages;
    char **cur_name = names;

    //두 번째 방법: 포인터 사용
    for(i = 0; i<count; i++_ {
        printf("%s is %d years old.\n", *(cur_name + i), *(cur_age + i));
    }

    printf("\n--\n");
```

```

//세 번째 방법: 포인터도 배열처럼 사용 가능
for(i = 0; i < count; i++){
    printf("%s is %d years old again.\n", cur_name[i], cur_age[i]);
}

printf("\n---\n");

//네 번째 방법: 복잡하고 어리석은 방식
for(cur_name = names, cur_age = ages; cur_age - ages < count; cur_name++,
cur_age++){
    printf("%s lived %d years so far.\n", *cur_name, *cur_age);
}

return 0;
}

```

[15-2] 실행 결과 예시

```

$ ./ex15
Alan has 23 years alive.
Frank has 43 years alive.
Mary has 12 years alive.
John has 89 years alive.
Lisa has 2 years alive.
---
Alan is 23 years old.
Frank is 43 years old.
Mary is 12 years old.
John is 89 years old.
Lisa is 2 years old.
---
Alan is 23 years old again.
Frank is 43 years old again.
Mary is 12 years old again.
John is 89 years old again.
Lisa is 2 years old again.
---
Alan lived 23 years so far.
Frank lived 43 years so far.

```

Mary lived 12 years so far.
John lived 89 years so far.
Lisa lived 2 years so far.

각 출력 줄이 코드의 어느 줄에서 나온 것인지 추적하자.

[15-3] 포인터 설명

ages[i]는 배열 ages의 i번째 요소를 의미한다.
i는 인덱스이며 배열 내부의 위치이다.

그렇다면 ages는 무엇인가?
C에서 ages는 배열이 시작되는 메모리 주소이다.
즉, ages는 배열의 첫 번째 정수가 있는 주소이다.

중요한 사실은 C는 여러분의 컴퓨터 전체를 하나의 거대한 바이트 배열로 생각한다. 그리고 그 위에 타입 개념을 덧씌운다.

C가 배열을 다루는 방식

- 메모리 블록 생성
- 배열 이름을 그 시작 주소에 연결
- 기본 주소 + 오프셋(i) 계산
- 올바른 타입 크기로 변환하여 값 변환

포인터란 무엇인가?

- 포인터는 메모리 어딘가를 가리키는 주소이다. 그리고 어떤 타입을 가리키는지도 알고 있다.

포인터는

- 증가 가능
- 감소 가능
- 더하기/빼기 가능
- 값 읽기 가능
- 값 쓰기 가능

포인터의 목적은 배열로는 부족할 때 직접 메모리를 조작하기 위함이다.
대부분의 경우 배열을 사용하고 포인터는 꼭 필요할 때만 사용하자.

[15-4] 포인터는 배열이 아니다

포인터와 배열은 같은 것이 아니다.

예:

`sizeof(cur_age)`

이것은 포인터의 크기이다.

배열 전체의 크기가 아니다. 배열 전체 크기를 얻으려면 배열 이름을 사용해야 한다.

[15-5] 망가뜨려 보기(How to Break It)

- `cur_age` 가 `names` 를 가리키게 만들어 보자.
- 마지막 `for-loop` 의 계산을 일부러 틀리게 해보자.
- 배열의 끝에서 시작해 거꾸로 순회해보자.

[15-6] 추가 과제(Extra Credit)

- 모든 배열을 포인터로 바꿔보자.
- 모든 포인터를 배열로 바꿔보자.
- 다른 프로그램에서 배열 대신 포인터를 사용해 보자.
- 값과 주소 연산을 조합해보자.
- `%p` 를 사용해 포인터 주소를 출력하자.
- 각 출력 방식을 함수로 분리하자.
- `for-loop` 을 `while-loop` 로 바꿔보자.

Exercise 16. 구조체와 그에 대한 포인터

이번 장에서는 struct 를 만드는 방법, 그 구조체를 가리키는 포인터를 만드는 방법, 그리고 이를 사용해 내부 메모리 구조를 이해하는 방법을 배운다.

또한 이전 연습에서 배운 포인터 지식을 적용하고, malloc 을 사용해 원시 메모리(raw memory)로부터 이러한 구조체를 생성하는 방법도 배우게 된다.

늘 그렇듯이, 먼저 프로그램을 직접 입력하고 실행되도록 만든다.

[16-1] ex16.c

```
#include <stdio.h>
#include <assert.h>
#include <stdlib.h>
#include <string.h>

struct Person{
    char *name;
    int age;
    int height;
    int weight
};

struct Person *Person_create(char *name, int age, int height, int weight)
{
    struct Person *who = malloc(sizeof(struct Person));
    assert(who != NULL);

    who->name = strdup(name);
    who->age = age;
    who->height = height;
    who->weight = weight;

    return who;
}

void Person_destroy(struct Person *who)
{
    assert(who != NULL);
    free(who->name);
    free(who);
}
```

```

}

void Person_print(struct Person *who)
{
    printf("Name: %s\n", who->name);
    printf("WtAge: %d\n", who->age);
    printf("WtHeight: %d\n", who->height);
    printf("WtWeight: %d\n", who->weight);
}

int main(int argc, char *argv[])
{
    struct Person *joe = Person_create("Joe Alex", 32, 64, 140);
    struct Person *frank = Person_create("Frank Blank", 20, 72, 180);

    printf("Joe is at memory location: %p\n", joe);
    Person_print(joe);

    printf("Frank is at memory location: %p\n", frank);
    Person_print(frank);

    joe->age +=20;
    joe->height -=2;
    joe->weight += 40;
    Person_print(joe);

    frank->age +=20;
    frank->height +=20;
    Person_print(frank);

    Person_destroy(joe);
    Person_destroy(frank);
    return 0;
}

```

[16-2] 프로그램 설명 방식

이번에는 줄 단위 설명을 하지 않는다.

대신, 프로그램의 구성 요소를 설명할테니, 각 줄이 무엇을 하는지 직접 적어보자.

includes

여기서 새로운 헤더 파일들을 포함한다.

각 헤더 파일이 무엇을 제공하는지 스스로 찾아보자.

struct Person

사람을 표현하기 위한 4 개의 요소를 가진 구조체를 만든다.

- naem
- age
- height
- weight

이 구조체는 하나의 새로운 복합 타입이다.

이는 데이터베이스 테이블의 한 행(row)이나 객체지향 언어(OOP)의 간단한 클래스와 비슷하다.

funcion Person_create

구조체를 생성하기 위한 함수이다.

중요한 점:

- malloc 을 사용해 OS 에 메모리 할당 요청
 - sizeof(struct Person)으로 구조체 전체 크기 계산
 - assert 로 malloc 이 NULL 을 반환하지 않았는지 확인
 - who->field 문법으로 각 필드 초기화
 - strdup 으로 문자열을 복사하여 구조체가 소유하도록 만들
- strdup 역시 내부적으로 malloc 을 사용한다.

function Person_destroy

create 가 있다면 반드시 destroy 도 있어야 한다.

- assert 로 유효성 검사
- free 로 malloc 과 strdup 으로 할당한 메모리 반환
- 하지 않으면 메모리 누수 발생

function Person_print

구조체 내용을 출력하는 함수이다.

x->y 문법을 사용해 각 필드에 접근한다.

function main

main에서는 다음을 수행한다.

- 두 사람 joe, frank 생성
- %p를 사용해 구조체의 메모리 위치 출력
- 두 사람을 20살씩 나이 증가
- 다시 출력
- 마지막에 destroy 호출하여 메모리 정리

해야 할 것

- 모르는 함수와 헤더를 모두 찾아보기
- 모든 줄 위에 영어로 설명 주석 달기
- 함수 호출과 변수 흐름 추적
- 이해 안되는 심볼 찾기

[16-3] 찾아본 것

[1] #include 헤더들의 집합

<stdio.h>

- 표준 입출력 관련 함수들이 들어있다.(printf(), scanf(), fopen(), fclose(), FILE 타입 등)

<assert.h>

- 디버깅용 검증 도구

프로그램 실행 중에 “이 조건은 반드시 참이어야 한다”라고 검사하는 용도이다.
조건이 거짓이면 프로그램이 즉시 종료된다.

예:

```
int x = 5;
assert(x == 5);
assert(x == 10);
```

보통 포인터 NULL 검사할 때 많이 사용된다.

<stdlib.h>

- 메모리 동적 할당과 관련된 핵심 헤더
- malloc(), calloc(), realloc(), free()
- exit()
- atoi(), rand() 등

<string.h>

- 문자열 복사, 비교, 길이 측정 등에 사용
- strlen(), strcpy(), strcmp(), strdup()

[2] assert 설명

```
assert(expression);
```

- expression 이 참(true): 그냥 지나감
- expression 이 거짓(false): 프로그램 강제 종료

예:

```
int *ptr = malloc(sizeof(int));  
assert(ptr != NULL);
```

- 메모리 할당 실패하면 프로그램을 멈춰서 버그를 빨리 찾게 해준다.
- 릴리즈 모드에서 #define NDEBUG 하면 assert 이 아예 제거된다.

[3] strdup 설명

```
strdup() = string duplicate  
char *copy = strdup("hello");
```

동작 과정:

1. 문자열 길이 계산
2. malloc()으로 그 길이만큼 메모리 할당
3. 문자열 복사
4. 새로 할당된 메모리 주소 반환

[16-4] 실행 결과 예시

```
$ ./ex16  
Joe is at memory location 0xeba010:  
Name: Joe Alex  
Age: 32  
Height: 64  
Weight: 140  
Frank is at memory location 0xeba050:  
Name: Frank Blank  
Age: 20  
Height: 72  
Weight: 180  
Name: Joe Alex  
Age: 52  
Height: 62  
Weight: 180  
Name: Frank Blank  
Age: 40  
Height: 72  
Weight: 200
```

[16-5] 구조체 설명

구조체는 여러 데이터 타입을 하나의 메모리 블록에 모아 둔 것이다.

특징

- 각각의 필드 타입을 가진다.
- C는 이를 하나의 메모리 블록에 배치
- struct Person은 새로운 복합 데이터 타입
- 함수에 통째로 전달 가능
- 포인터일 경우 $x \rightarrow y$
- 포인터가 아닐 경우 $x.y$ 사용

구조체가 없다면

- 메모리 크기 계산
- 위치 계산
- 직접 주소 계산

을 모두 수동으로 해야 한다.

[16-6] 망가뜨려 보기(How to Break It)

- Person_destroy(NULL) 호출
- destroy 호출을 빼고 디버거 실행 → 메모리 누수 확인
- $\text{who} \rightarrow \text{name}$ free 제거
- Person_print(NULL) 호출 → 프로그램 크래시

[16-7] 추가 과제(Extra Credit)

이 프로그램을 포인터와 malloc 없이 다시 작성해 보자.

- 스택에 구조체 생성 방법
- $x.y$ 문법 사용법
- 포인터 없이 구조체를 함수에 전달하는 방법

Exercise 17. 힙과 스택 메모리 할당

이번 장에서는 난이도가 크게 상승된다. 작은 데이터베이스 프로그램 전체를 직접 만들게 된다. 이 데이터베이스는 효율적이지도 않고 많은 데이터를 저장하지도 못하지만, 지금까지 배운 대부분의 내용을 보여준다.

또한:

- 메모리 할당을 보다 공식적으로 소개하고 파일 작업을 시작하게 된다.

파일 I/O 함수들을 사용하지만, 자세히 설명하지는 않을 것이다.

여러분이 먼저 스스로 이해해보기를 바란다.

늘 그렇듯이, 전체 프로그램을 직접 입력하고 실행되도록 만든 후에 설명을 읽어보자.

[17-1] ex17.c

```
#include <stdio.h>
#include <assert.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>

#define MAX_DATA 512
#define MAX_ROWS 100

struct Address {
    int id;
    int set;
    char name[MAX_DATA];
    char email[MAX_DATA];
};

struct Database {
    struct Address rows[MAX_ROWS];
};

struct Connection {
    FILE *file;
    struct Database *db;
};

void die(const char *message)
```



```

{
    if (errno) {
        perror(message);
    } else {
        printf("ERROR: %s\n", message);
    }

    exit(1);
}

void Address_print(struct Address *addr)
{
    printf("%d %s %s\n", addr->id, addr->name, addr->email);
}

void Database_load(struct Connection *conn)
{
    int rc = fread(conn->db, sizeof(struct Database), 1, conn->file);
    if (rc != 1)
        die("Failed to load database.");
}

struct Connection *Database_open(const char *filename, char mode)
{
    struct Connection *conn = malloc(sizeof(struct Connection));
    if (!conn)
        die("Memory error");

    conn->db = malloc(sizeof(struct Database));
    if (!conn->db)
        die("Memory error");

    if (mode == 'c') {
        conn->file = fopen(filename, "w");
    } else {
        conn->file = fopen(filename, "r+");

        if (conn->file) {
            Database_load(conn);
        }
    }
}

```

```

        if (!conn->file)
            die("Failed to open the file");

        return conn;
    }

void Database_close(struct Connection *conn)
{
    if (conn) {
        if (conn->file)
            fclose(conn->file);
        if (conn->db)
            free(conn->db);
        free(conn);
    }
}

void Database_write(struct Connection *conn)
{
    rewind(conn->file);

    int rc = fwrite(conn->db, sizeof(struct Database), 1, conn->file);
    if (rc != 1)
        die("Failed to write database.");

    rc = fflush(conn->file);
    if (rc == -1)
        die("Cannot flush database.");
}

void Database_create(struct Connection *conn)
{
    int i = 0;

    for (i = 0; i < MAX_ROWS; i++) {
        struct Address addr = {.id = i, .set = 0};
        conn->db->rows[i] = addr;
    }
}

```

```

void Database_set(struct Connection *conn, int id,
                  const char *name, const char *email)
{
    struct Address *addr = &conn->db->rows[id];
    if (addr->set)
        die("Already set, delete it first");

    addr->set = 1;

    char *res = strncpy(addr->name, name, MAX_DATA);
    if (!res)
        die("Name copy failed");

    res = strncpy(addr->email, email, MAX_DATA);
    if (!res)
        die("Email copy failed");
}

void Database_get(struct Connection *conn, int id)
{
    struct Address *addr = &conn->db->rows[id];

    if (addr->set) {
        Address_print(addr);
    } else {
        die("ID is not set");
    }
}

void Database_delete(struct Connection *conn, int id)
{
    struct Address addr = {.id = id, .set = 0};
    conn->db->rows[id] = addr;
}

void Database_list(struct Connection *conn)
{
    int i = 0;
    struct Database *db = conn->db;

    for (i = 0; i < MAX_ROWS; i++) {

```

```

        struct Address *cur = &db->rows[i];

        if (cur->set) {
            Address_print(cur);
        }
    }
}

int main(int argc, char *argv[])
{
    if (argc < 3)
        die("USAGE: ex17 <dbfile> <action> [action params]");

    char *filename = argv[1];
    char action = argv[2][0];
    struct Connection *conn = Database_open(filename, action);

    int id = 0;

    if (argc > 3)
        id = atoi(argv[3]);

    if (id >= MAX_ROWS)
        die("There's not that many records.");

    switch (action) {
        case 'c':
            Database_create(conn);
            Database_write(conn);
            break;

        case 'g':
            if (argc != 4)
                die("Need an id to get");

            Database_get(conn, id);
            break;

        case 's':
            if (argc != 6)
                die("Need id, name, email to set");
    }
}

```

```

        Database_set(conn, id, argv[4], argv[5]);
        Database_write(conn);
        break;

    case 'd':
        if (argc != 4)
            die("Need id to delete");

        Database_delete(conn, id);
        Database_write(conn);
        break;

    case 'l':
        Database_list(conn);
        break;

    default:
        die("Invalid action: c=create, g=get, s=set, d=del, l=list");
}

Database_close(conn);

return 0;
}

```

[17-2] 프로그램 개요

이 프로그램에서는 여러 개의 struct 를 사용해 간단한 주소록 데이터베이스를 만든다.

[1] #define 을 사용한 상수 정의

```

#define MAX_DATA 512
#define MAX_ROWS 100

```

C 전처리기(CPP)를 사용해 상수를 정의한다. 이 방법은 신뢰성 있는 상수 정의 방식이다. 다른 방법도 있지만, 특정 상황에서는 사용할 수 없다.

[2] 고정 크기 구조체(Fixed Sized Structs)

struct Address 는:

- 고정 크기의 name 배열
- 고정 크기의 email 배열

을 가진다.

이 방식은 비효율적이지만 디스크에 저장하고 읽기 쉽다.

struct Database 는 Address 구조체 배열을 가지므로 역시 고정 크기이다.

그래서 나중에 파일에 한 번에 전체를 fwrite 할 수 있다.

[3] die 함수

작은 프로그램에서는 에러가 발생하면 종료시키는 단일 함수를 만들 수 있다.

die()는

- 에러 메시지 출력
- 프로그램 종료

를 수행한다.

[4] errno 와 perror

함수가 실패하면 errno 라는 외부 변수를 설정한다.

- errno 는 숫자
- perror 는 사람이 읽을 수 있는 에러 메시지 출력

[5] FILE 관련 함수

새로운 파일 함수 등장

- fopen
- fread
- fwrite
- fclose
- rewind

이 함수들은 FILE 구조체를 사용한다. 이 구조체는 C 표준 라이브러리에서 정의되어 있다.

[6] 중첩 구조체 포인터

다음 코드를 보자.

&conn->db->rows[i]

- conn 안의 db 안의 rows 배열의 i 번째 요소의 주소를 가져온다.

[7] 구조체 프로토타입 복사

Database_delete 함수에서

```
struct Address addr = {id = id, .set = 0};  
conn->db->rows[id] = addr;
```

임시 구조체를 만든 뒤 원하는 위치에 그대로 대입한다.
C에서는 구조체를 그냥 대입하면 자동으로 복사된다.
memcpy를 사용할 필요가 없다.

[8] 복잡한 인자 처리

main()에서 명령행 인자를 복잡하게 처리한다.
더 좋은 방법은 나중에 배우게 된다.

[9] 문자열을 정수로 변환

```
atoi()
```

문자열을 정수로 변환한다.

[10] 대량 데이터 힙 할당

이 프로그램의 핵심은

```
malloc(sizeof(struct Database))
```

큰 데이터를 힙에 할당하는 것이다.

[11] NULL은 0이 아니다.

```
if(!ptr) die("fail!");
```

NULL은 false로 평가된다.

표준상 NULL은 0처럼 취급 가능하다.

[17-3] 실행 결과 예시

```
$ ./ex17 db.dat c  
$ ./ex17 db.dat s 1 zed zed@zedshaw.com  
$ ./ex17 db.dat s 2 frank frank@zedshaw.com  
$ ./ex17 db.dat s 3 joe joe@zedshaw.com
```

```
$ ./ex17 db.dat l  
1 zed zed@zedshaw.com  
2 frank frank@zedshaw.com  
3 joe joe@zedshaw.com
```

```
$ ./ex17 db.dat d 3
```

```
$ ./ex17 db.dat l
1 zed zed@zedshaw.com
2 frank frank@zedshaw.com

$ ./ex17 db.dat g 2
2 frank frank@zedshaw.com
```

[17-4] 힙과 스택 메모리 할당

요즘 언어들은 메모리 위치를 신경 쓰지 않아도 된다. 하지만 C는 다르다.
C는 실제 CPU 메모리 구조를 사용한다.

- Stack
- Heap

Heap

- 컴퓨터의 남은 메모리 공간
- malloc()으로 할당
- free()로 반환
- 반환하지 않으면 메모리 누수

Valgrind가 누수 추적 도움

Stack

- 함수의 지역 변수 저장 공간
- 함수 호출 시 push
- 함수 종료 시 pop

장점:

- 자동 정리
- 메모리 누수 방지

[17-5] 스택과 힙에서 발생할 수 있는 문제

1. malloc으로 얻은 포인터가 스택에 있다가 함수 종료 → 포인터 사라짐
2. 스택에 너무 큰 데이터 저장 → 스택 오버플로우
3. 스택 변수의 주소를 반환 → 세그멘테이션 오류

[17-6] Database_open / Database_close 를 만들었는가

- open은 메모리 전체 생성
- close는 전체 정리

create/destroy 패턴은 메모리 관리에 매우 중요하다.

[17-7] 망가뜨려 보기(How to Break It)

- 레코드 번호 체크 제거
- 데이터 파일을 직접 편집해 깨뜨리기
- 잘못된 명령행 인자 전달
- strncpy 버그 찾기(512바이트 초과 시 NULL 보장 안됨 → 마지막에 'W0' 강제 삽입)
- 힙 메모리 부족 발생시켜 보기

[17-8] 추가 과제(Extra Credit)

- die 함수에서 conn 정리하도록 수정
- MAX_DATA, MAX_ROWS 를 동적으로 변경 가능하게 수정
- find 기능 추가
- struct 패킹 방식 조사
- Address 에 필드 추가
- 자동 테스트 셸 스크립트 작성(set -e)
- 전역 변수 방식으로 재작성
- 스택 자료구조 직접 구현

[17-9] 찾아본 것

[1] fread() 기본 개념

fread()는 파일에서 바이너리 데이터를 읽어오는 함수이다.

```
size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);
```

[1-1] 매개변수 설명

인자	의미
ptr	읽은 데이터를 저장할 메모리 주소
size	한 덩어리의 크기 (바이트 단위)
nmemb	몇 개를 읽을지
stream	파일 포인터

[1-2] 코드 해석

```
inc rc = fread(conn->db, sizeof(struct Database), 1, conn->file);
```

conn->file 에서 struct Database 크기만큼 1 개를 읽어서 conn->db 메모리에 저장한다.

[1-3] 반환값 rc 의 의미

fread()는 실제로 읽은 개수(nmemb 단위)를 반환한다.

[1-4] fread 는 바이너리 단위로 읽는다.

fread()는 텍스트가 아니라 메모리 그대로 읽는다.

예를 들어:

```
fwrite(db, sizeof(struct Database), 1, file);
```

로 저장했다면, 그 메모리 구조가 그대로 파일에 써진다.

그렇기에 로드할 때도:

```
fread(db, sizeof(struct Database), 1, file);
```

로 동일한 크기로 읽어야 한다 .

[1-5] 왜 sizeof(struct database)를 쓰는가

- 구조체 전체 크기만큼 정확히 읽어올 수 있다.
- 하드코딩된 숫자 쓰면 위험하다.

[1-6] 주의점

이 방식은 다음 조건에서 안전하다.

1. 구조체 내부에 포인터가 없어야 함
2. 컴파일러 패딩 차이가 없어야 함
3. 다른 컴퓨터에서 구조체 정렬 방식이 달라지지 않아야 함

[2] rewind()

```
void rewind(FILE *stream);
```

[2-1] 기능

파일 위치 포인터를 파일의 처음(0 번 위치)으로 이동시킨다.

[2-2] 내부 동작

```
fseek(stream, 0, SEEK_SET);
```

```
clearerr(stream);
```

- 파일 위치를 처음으로 이동
- 에러 및 EOF 상태 초기화

[2-3] 반환 값

- 없음(void)

[2-4] 사용 목적

- 파일을 다시 처음부터 읽거나
- 기존 파일을 덮어쓰기 위해 위치를 초기화할 때 사용

[3] fflush()

```
int fflush(FILE *stream);
```

[3-1] 기능

- 출력 버퍼에 남아 있는 데이터를 즉시 파일(디스크)에 기록한다.

[3-2] 반환값

- 성공: 0
- 실패: EOF(보통 -1)

[3-3] 사용 목적

- 데이터 유실 방지
- 즉시 파일 저장 보장
- 로그 기록 등 실시간 반영 필요할 때

[4] strncpy()

```
char *strncpy(char *dest, const char *src, size_t n);
```

[4-1] 기능

- src 문자열을 최대 n 바이트까지 dest 에 복사한다.

[4-2] 매개변수

인자	의미
dest	복사될 목적지 버퍼
src	복사할 원본 문자열
n	최대 복사할 바이트 수

[4-3] 반환값

- 성공 시 → dest 의 주소 반환
- 실패 개념은 없음(NULL 반환 거의 없음)

[4-4] 동작 방식

src 길이 < n 인 경우

```
src = "abc"
n = 10
```

결과:

```
a b c \0 \0 \0 \0 \0 \0 \0
```

남은 공간을 \0 으로 채운다.

src 길이 $\geq n$ 인 경우

```
src = "abcdefgh"
n = 5
```

결과:

```
a b c d e
```

자동으로 'W0'을 붙이지 않는다.

[4-5] 가장 중요한 특징

strncpy()는 항상 null-terminated 문자열을 보장하지 않는다.

그래서 안전하게 쓰려면 다음과 같이 쓴다.

```
strncpy(dest, src, MAX_DATA - 1);
dest[MAX_DATA - 1] = 'W0';
```

[17-10] 파일 기반 구조체 데이터베이스 프로그램 분석 및 설계 사고 과정

Exercise 18. 함수 포인터

C에서 함수는 실제로 프로그램 안의 어떤 코드가 존재하는 위치를 가리키는 포인터일 뿐이다.

지금까지 구조체, 문자열, 배열에 대한 포인터를 만들었던 것처럼, 함수에도 포인터를 가리킬 수 있다. 이것의 주요 사용 목적은 다른 함수에 콜백(callback)을 전달하거나, 클래스와 객체를 흉내 내기 위함이다.

이 장에서는 콜백을 사용하고, 다음 장에서는 간단한 객체 시스템을 만들 것이다.

[18-1] ex18.c

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>

/*ex17에서 사용했던 die 함수*/
void die(const char *message)
{
    if (errno) {
        perror(message);
    } else {
        printf("ERROR: %s\n", message);
    }

    exit(1);
}

/*
typedef는 새로운 가짜 타입을 만든다.
여기서는 함수 포인터 타입을 만든 것이다.
*/
typedef int (*compare_cb)(int a, int b);

/*
compare_cb를 사용하여 정렬을 수행하는 전형적인 버블 정렬 함수
*/
int *bubble_sort(int *numbers, int count, compare_cb cmp)
{
    int temp = 0;
    int i = 0;
```

```

    int j = 0;

    int *target = malloc(count * sizeof(int));
    if (!target) die("Memory error.");

    memcpy(target, numbers, count * sizeof(int));

    for (i = 0; i < count; i++) {
        for (j = 0; j < count - 1; j++) {
            if (cmp(target[j], target[j + 1]) > 0) {
                temp = target[j + 1];
                target[j + 1] = target[j];
                target[j] = temp;
            }
        }
    }

    return target;
}

int sorted_order(int a, int b)
{
    return a - b;
}

int reverse_order(int a, int b)
{
    return b - a;
}

int strange_order(int a, int b)
{
    if (a == 0 || b == 0) {
        return 0;
    } else {
        return a % b;
    }
}

/**
 * Used to test that we are sorting things correctly

```

```

    * by doing the sort and printing it out.
    */
void test_sorting(int *numbers, int count, compare_cb cmp)
{
    int i = 0;

    int *sorted = bubble_sort(numbers, count, cmp);
    if (!sorted) die("Failed to sort as requested.");

    for (i = 0; i < count; i++) {
        printf("%d ", sorted[i]);
    }

    printf("\n");

    free(sorted);
}

int main(int argc, char *argv[])
{
    if (argc < 2) die("USAGE: ex18 4 3 1 5 6");

    int count = argc - 1;
    int i = 0;
    char **inputs = argv + 1;

    int *numbers = malloc(count * sizeof(int));
    if (!numbers) die("Memory error.");

    for (i = 0; i < count; i++) {
        numbers[i] = atoi(inputs[i]);
    }

    test_sorting(numbers, count, sorted_order);
    test_sorting(numbers, count, reverse_order);
    test_sorting(numbers, count, strange_order);

    free(numbers);

    return 0;
}

```

[18-2] 함수 포인터 형식

함수 포인터란 함수의 주소를 저장하는 포인터 변수이다.

C에서 함수 이름은 해당 함수의 시작 주소를 의미한다.
따라서 함수도 일반 변수처럼 포인터에 저장할 수 있다.

[1] 기본 함수 선언

```
int callme(int a, int b);
```

- int 를 반환
- 매개변수는 (int, int)

[2] 함수 포인터 선언 방법

단계별 작성 방법

[2-1] 일반 함수 선언을 작성한다.

```
int callme(int a, int b);
```

[2-2] 함수 이름을 괄호로 감싸고 앞에 *를 붙인다.

```
int (*callme) (int a, int b);
```

[2-3] 함수 이름을 포인터 변수 이름으로 변경한다.

```
int (*compare_cb) (int a, int b)
```

[3] 읽는 방법

```
int (*compare_cb) (int a, int b);
```

1. 이름부터 본다: compare_cb
2. (*compare_cb): 포인터
3. (int a, int b): int 두 개를 받는 함수
4. int 를 반환한다.

즉, compare_cb 는 int 두 개를 받아서 int 를 반환하는 함수의 주소를 저장하는 포인터이다.

[4] 함수처럼 사용하기

```
int (*tester) (int a, int b_ = sorted_order;  
printf("%d\n", tester(2, 3));
```

- tester 는 sorted_order 함수의 주소를 저장
- tester(2, 3)은 내부적으로 (*tester)(2, 3)과 동일
- 즉, 일반 함수처럼 호출 가능

[5] 괄호의 중요성

다음 두 선언은 완전히 다르다.

함수 포인터

```
int (*f)(int);
```

- f는 함수 포인터
- int를 받아 int를 반환하는 함수의 주소를 지정

포인터를 반환하는 함수

```
int *f(int);
```

- f는 함수
- int를 받아 int*를 반환

[6] 포인터를 반환하는 함수의 함수 포인터

일반 함수:

```
char *make_coolness(int awesome_levels);
```

- int를 받아 char*를 반환

함수 포인터로 만들면:

```
char *(*coolness_cb)(int awesome_levels);
```

- coolness_cb는 포인터
- int를 받는 함수
- char*를 반환하는 함수의 주소를 저장

[18-3] 함수 포인터를 매개변수로 전달하는 문제

[1] 문제 상황

함수 포인터는 다른 함수에 매개변수로 전달할 수 있다.

이때 주로 콜백 함수(callback function)를 전달하는 경우가 많다.

하지만 함수 포인터 문법은 복잡해서 직접 작성하면 가독성이 매우 떨어진다.

[2] 기본 예시(typedef 없이 작성)

예를 들어, 다음과 같은 함수가 있다고 가정한다.

```
int compare(int a, int b)
```

이 함수를 다른 함수의 매개변수로 전달하려면 다음과 같이 작성해야 한다.

```
void sort(int *numbers, int count, int(*compare_cb)(int a, int b));
```

- compare_cb 는 함수 포인터
- int 두 개를 받아
- int 를 반환하는 함수의 주소를 저장

문법 자체는 맞지만, 선언이 길어질수록 매우 복잡해진다.

[3] 해결 방법: typedef 사용

[3-1] typedef 란

typedef 는 C 에서 기존 타입에 새로운 이름을 붙이는 키워드이다.

복잡한 타입을 단순한 이름으로 바꿔 가독성을 높일 수 있다.

[4] 함수 포인터에 typedef 적용하기

[4-1] 기존 함수 포인터 선언

```
int (*compare_cb)(int a, int b);
```

[4-2] 앞에 typedef 추가

```
typedef int (*compare_cb)(int a, int b);
```

이제 compare_cb 는 하나의 타입이 된다.

[5] typedef 적용 후 매개변수 전달

이제 함수는 훨씬 간단해진다.

```
void sort(int *numbers, int count, compare_cp cmp);
```

- cmp 는 compare_cb 타입
 - 즉, int 두 개를 받아 int 를 반환하는 함수 포인터
- 가독성이 크게 개선된다.

[18-4] 실행 결과 예시

```
$ make ex18
cc -Wall -g ex18.c -o ex18
$ ./ex18 4 1 7 3 2 0 8
0 1 2 3 4 7 8
8 7 4 3 2 1 0
3 4 2 7 1 0 8
$
```

[18-5] 망가뜨려 보기(How to Break It)

함수 포인터도 다른 포인터처럼 메모리를 가리킨다.

C에서는 한 포인터를 다른 포인터 타입으로 변환할 수 있다.

보통 필요하지 않지만, 해킹이 어떻게 가능한지 보여주기 위해 test_sorting 끝에 다음을 추가하자.

```
unsigned char *data = (unsigned char *)cmp;
for(i = 0; i < 25; i++) {
    printf("%02x:", data[i]);
}
printf("\n");
```

이 반복문은 함수를 문자열처럼 변환해서 내용을 출력하는 것과 비슷하다.

CPU 나 OS 가 이것을 막지 않는 한 프로그램은 깨지지 않는다.

정렬된 배열을 출력한 후 다음과 같은 16 진수 숫자들이 출력된다.

```
55:48:89:e5:89:7d:fc:89:75:f8:8b:55:fc:8b:45:
```

이것은 해당 함수의 실제 어셈블리 바이트 코드다.

시작 부분은 동일하지만 끝 부분은 다를 것이다.

또한 이 반복문이 함수 전체를 읽지 못했을 수도 있고 다른 메모리를 침범했을 수도 있다.

추가 분석 없이는 알 수 없다.

[18-6] 추가 과제(Extra Credit)

이 프로그램을 포인터와 malloc 없이 다시 작성해 보자.

- hexs 에디터로 ex18 을 열고 함수 시작 바이트를 찾아보라.
- 다른 무작위 값도 변경해보고 다시 실행해보자.
- compare_cb 에 잘못된 함수를 전달해보고 컴파일러 에러를 확인하자.
- NULL 을 전달해보고 프로그램이 어떻게 망가지는지 확인하자.
- 다른 정렬 알고리즘을 작성하고 test_sorting 이 정렬 함수와 비교 함수를 모두 받도록 수정하자.

Exercise 19. Zed 의 끝내주는 디버그 매크로

C 에는 우리가 계속 피해 다녀왔던 반복되는 문제가 하나 있다.

이번 연습에서 내가 만든 매크로 세트를 사용해서 그 문제를 해결할 것이다.

나중에 이 매크로들이 얼마나 엄청나게 대단한지 깨닫게 되면 나에게 고마워하게 될 것이다.
지금은 얼마나 대단한지 모르겠지만, 일단 사용해봐라

그러면 언젠가 나를 찾아와 이렇게 말하게 될 것이다.

“Zed, 그 디버그 매크로들은 진짜 최고였어요. 덕분에 10 년 치 고통을 피했고, 여러 번
자살할 뻔한 걸 막아줬어요.

감사합니다. 여기 백만 달러와 Leo Fender 가 사인한 Snakehead Telecaster 원형을 드리죠”

이 정도로 대단한 것이다.

[19-1] C 의 에러 처리 문제

에러를 처리하는 일은 거의 모든 프로그래밍 언어에서 어려운 작업이다.

어떤 언어들은 아예 “에러”라는 개념 자체를 피하려고 한다.

다른 언어들은 예외(exception)같은 복잡한 제어 구조를 만들어 에러를 전달한다.

문제의 근본 원인은 프로그래머들이 “에러는 안 일어난다”고 가정하기 때문이다.

이 낙관주의가 언어 설계에까지 영향을 준다.

C 는 함수가 에러 코드를 반환하고, 전역 변수 `errno` 를 설정하는 방식으로 문제를 해결한다.
그래서 매번 이것을 확인해야 한다.

그 결과 코드는 이렇게 된다.

- 함수 호출
- 반환값이 에러인지 검사
- 지금까지 생성한 자원들을 정리
- 도움이 될 만한 에러 메시지 출력

즉, 함수 호출 하나당 3~4 줄 더 써야 한다. 여기에 정리 코드까지 포함하면 더 길어진다.
예를 들어, 구조체 10 개, 파일 3 개, DB 연결 1 개가 있다면 에러 발생 시 14 줄 이상이
추가된다.

과거에는 이게 문제가 아니었다. C 프로그램은 에러가 나면 그냥 죽었기 때문이다. OS 가
정리를 대신 해줬다.

하지만 요즘 C 프로그램은 몇 주, 몇 달, 몇 년 동안 실행된다.
웹 서버가 작은 에러에 죽으면 안 된다. 라이브러리가 전체 프로그램을 죽여버리면 안된다.

다른 언어는 예외로 해결하지만, C에서는 문제가 있다. C는 반환 값이 하나 뿐이다.
예외는 스택 기반의 복잡한 시스템을 만든다.
C에서 예외를 흉내 내는 건 어렵고, 다른 라이브러리도 이해 못한다.

[19-2] 디버그 매크로

내가 수년간 사용해온 해결책은 작은 디버그 매크로 세트다. 이것은 기본적인 디버깅 및
에러 처리 시스템을 제공한다.

원리:

에러가 발생하면 함수는 error: 라벨로 점프한다.
그 부분에서 모든 정리를 수행하고 에러 코드를 반환한다.

check 라는 매크로로 조건을 검사하고,
에러 메시지를 출력하고, error:로 점프한다.

로그 함수들과 함께 사용하면 디버깅 메시지도 출력할 수 있다.
이제 내가 만든 최고의 걸작을 보여준다.

[19-3] dbg.h 코드

```
#ifndef __dbg_h__
#define __dbg_h__
/*
 * 헤더 파일이 여러 번 포함되는 것을 방지하기 위한 가드(guard)
 * __dbg_h__가 정의되어 있지 않으면 정의하고, 내용 포함
 */

#include <stdio.h> // fprintf, stderr 등 사용
#include <errno.h> // errno 사용
#include <string.h> // strerror 사용

/*
 * NDEBUG 가 정의되어 있으면 debug 매크로를 제거한다.
 * (No Debug 모드)
 */
#endif NDEBUG
```

```

/*
 * NDEBUG 가 정의된 경우 debug 는 아무 동작도 하지 않음
 * → 완전히 제거됨
 */
#define debug(M, ...)

#else

/*
 * debug 매크로 정의
 * M 은 포맷 문자열
 * ... 는 가변 인자 (printf 처럼 사용 가능)
 *
 * __FILE__ : 현재 파일 이름
 * __LINE__ : 현재 줄 번호
 * ##__VA_ARGS__ : 가변 인자를 이 위치에 삽입
 */
#define debug(M, ...) W
    fprintf(stderr, "DEBUG %s:%d: " M "Wn", W
        __FILE__, __LINE__, ##__VA_ARGS__)

#endif

/*
 * errno 를 사람이 읽기 좋은 문자열로 변환
 * errno 가 0 이면 "None"
 * 아니면 strerror(errno) 반환
 *
 * 삼항 연산자 사용:
 * (조건) ? 참일 때 값 : 거짓일 때 값
 */
#define clean_errno() (errno == 0 ? "None" : strerror(errno))

/*
 * 에러 로그 매크로
 * 사용자에게 보여줄 에러 메시지
 * debug 와 달리 컴파일 시 제거되지 않음
 */
#define log_err(M, ...) W

```

```

    fprintf(stderr, "[ERROR] (%s:%d: errno: %s) " M "Wn", W
        __FILE__, __LINE__, clean_errno(), ##__VA_ARGS__)

/*
 * 경고 로그 매크로
 */
#define log_warn(M, ...) W
    fprintf(stderr, "[WARN] (%s:%d: errno: %s) " M "Wn", W
        __FILE__, __LINE__, clean_errno(), ##__VA_ARGS__)

/*
 * 정보 로그 매크로
 * errno 는 출력하지 않음
 */
#define log_info(M, ...) W
    fprintf(stderr, "[INFO] (%s:%d) " M "Wn", W
        __FILE__, __LINE__, ##__VA_ARGS__)

/*
 * check 매크로
 *
 * A 가 참인지 검사
 * 거짓이면:
 *     1) log_err 로 메시지 출력
 *     2) errno 를 0 으로 초기화
 *     3) error 라벨로 점프
 *
 * 반드시 함수 안에 error: 라벨이 있어야 한다.
 */
#define check(A, M, ...) W
    if(!(A)) { W
        log_err(M, ##__VA_ARGS__); W
        errno = 0; W
        goto error; W
    }

/*

```

```

* sentinel 매크로
*
* 절대 실행되면 안 되는 코드 위치에 사용
* (예: switch 의 default)
*
* 실행되면:
*   에러 출력 후 error 로 점프
*/
#define sentinel(M, ...) W
    { W
        log_err(M, ##__VA_ARGS__); W
        errno = 0; W
        goto error; W
    }

/*
* 메모리 체크 전용 매크로
*
* 포인터 A 가 NULL 이면
* "Out of memory." 에러 출력 후 error 로 점프
*/
#define check_mem(A) W
    check((A), "Out of memory.")

/*
* check_debug 매크로
*
* check 와 거의 동일하지만,
* 에러 메시지를 log_err 대신 debug 로 출력
*
* NDEBUG 정의 시 debug 메시지는 제거되지만
* 검사와 goto error 는 그대로 동작
*/
#define check_debug(A, M, ...) W
    if(!(A)) { W
        debug(M, ##__VA_ARGS__); W
        errno = 0; W
        goto error; W
    }

```



```
#endif
/*
 * 헤더 가드 종료
 */
```

[19-4] dbg.h 의 모든 매크로를 실제 실행해보는 테스트 코드

```
#include "dbg.h"
#include <stdlib.h>
#include <stdio.h>

void test_debug()
{
    // notice you don't need the \n
    debug("I have Brown Hair.");

    // passing in arguments like printf
    debug("I am %d years old.", 37);
}

void test_log_err()
{
    log_err("I believe everything is broken.");
    log_err("There are %d problems in %s.", 0, "space");
}

void test_log_warn()
{
    log_warn("You can safely ignore this.");
    log_warn("Maybe consider looking at: %s.", "/etc/passwd");
}

void test_log_info()
{
    log_info("Well I did something mundane.");
    log_info("It happened %f times today.", 1.3f);
}

int test_check(char *file_name)
{

```

```

FILE *input = NULL;
char *block = NULL;

block = malloc(100);
check_mem(block); // should work

input = fopen(file_name, "r");
check(input, "Failed to open %s.", file_name);

free(block);
fclose(input);
return 0;

error:
    if (block) free(block);
    if (input) fclose(input);
    return -1;
}

int test_sentinel(int code)
{
    char *temp = malloc(100);
    check_mem(temp);

    switch (code) {
        case 1:
            log_info("It worked.");
            break;
        default:
            sentinel("I shouldn't run.");
    }

    free(temp);
    return 0;

error:
    if (temp) free(temp);
    return -1;
}

int test_check_mem()

```

```
{
    char *test = NULL;
    check_mem(test);

    free(test);
    return 1;

error:
    return -1;
}

int test_check_debug()
{
    int i = 0;
    check_debug(i != 0, "Oops, I was 0.");

    return 0;

error:
    return -1;
}

int main(int argc, char *argv[])
{
    check(argc == 2, "Need an argument.");

    test_debug();
    test_log_err();
    test_log_warn();
    test_log_info();

    check(test_check("ex19.c") == 0, "failed with ex19.c");
    check(test_check(argv[1]) == -1, "failed with argv");
    check(test_sentinel(1) == 0, "test_sentinel failed.");
    check(test_sentinel(100) == -1, "test_sentinel failed.");
    check(test_check_mem() == -1, "test_check_mem failed.");
    check(test_check_debug() == -1, "test_check_debug failed.");

    return 0;

error:
```

```
    return 1;
}
```

[19-5] 왜 die 함수 대신 매크로를 사용하는지

- file:line 정보를 얻기 위해
- goto error 로 정리 루틴 사용하기 위해
- 일반 if 문과 구분하기 위해
- 조건부 컴파일 가능하게 하기 위해

NDEBUG 를 정의하면 debug 매크로는 제거된다.

[19-6] 추가 과제(Extra Credit)

- 파일 상단에 #define NDEBUG cnrk
- Makefile 에 -DNDEBUG 추가
- 로그에 함수 이름도 포함하도록 수정

Exercise 19-1. Zed 의 끝내주는 디버그 매크로 상세 설명

[1] 헤더 가드(Header Guard)

```
#ifndef __dbg_h__
#define __dbg_h__
...
#endif
```

목적

- 이 파일이 여러 번 #include 되어도 한 번만 포함되도록 보장
- 중복 정의 방지

#ifndef(If Not Defined)

“만약 이 매크로가 정의되어 있지 않다면”이라는 의미이다.

```
#ifndef 매크로 이름
    // 매크로가 정의되어 있지 않을 때 실행될 코드
#endif
```

예시

```
#ifndef PI
#define PI 3.141592
#endif
```

- PI 가 아직 정의되지 않았다면 정의한다.
- 이미 정의되어 있다면 아무것도 하지 않는다.

[2] 디버그 매크로(debug)

NDEBUG 가 정의된 경우

```
#ifdef NDEBUG
#define debug(M, ...)
```

- 아무 동작도 하지 않음
- 컴파일 시 완전히 제거됨
- Release 모드에서 사용

#ifdef(If Defined)

“만약 이 매크로가 정의되어 있다면”이라는 의미

```
#ifdef 매크로 이름
    // 매크로가 정의되어 있을 때 실행될 코드
#endif
```

예시

```
#ifdef DEBUG
    printf("디버그 모드\n");
#else
    printf("릴리즈 모드\n");
#endif
```

NDEBUG 가 정의되지 않은 경우

```
#define debug(M, ...) W
    fprintf(stderr, "DEBUG %s:%d: " M "\n", W
        __FILE__, __LINE__, ##__VA_ARGS__)
```

출력 형식 예시

```
DEBUG main.c:25: x = 10
```

자동으로 포함되는 정보

매크로	의미
__FILE__	현재 파일 이름
__LINE__	현재 줄 번호
##__VA_ARGS__	가변 인자 처리

[3] 매크로란

매크로는 컴파일 전에 문자열을 치환하는 기능이다.

C 컴파일 과정은 다음과 같이 진행된다.

1. 전처리 (매크로 처리)
2. 컴파일
3. 링크

#define 은 전처리 단계에서 단순 치환이다.

[3-1] 가장 기본적인 매크로

값 치환 매크로

```
#define PI 3.14
```

사용:

```
printf("%f", PI);
```

전처리 후

```
printf("%f", 3.14);
```

[3-2] 함수처럼 보이는 매크로

```
#define SQUARE(x) (x * x)
```

사용:

```
int a = SQUARE(3);
```

전처리 후:

```
int a = (3 * 3);
```

[3-3] 여러 줄 매크로 만드는 법

줄 끝에 `₩`를 붙이면 된다.

```
#define PRINT_HELLO() ₩  
    printf("Hello₩n"); ₩  
    printf("World₩n");
```

[3-4] 가변 인자 매크로

```
#define 이름(고정인자, ...) 코드
```

`...` = 여러 개 인자 받기

`__VA_ARGS__` = 전달된 실제 값들

예제:

```
#define myprint(fmt, ...) printf(fmt, __VA_ARGS__)
```

사용:

```
myprint("값=%₩n", 10);
```

전처리 후:

```
printf("값=%d₩n", 10);
```

[4] `errno` 정리 매크로

```
#define clean_errno() (errno == 0 ? "None" : strerror(errno))
```

역할

- `errno == 0` : "None"

- 아니면 : `strerror(errno)` 반환

즉, 사람이 읽을 수 있는 에러 메시지로 변환

[5] 로그 매크로들

[5-1] log_err

```
#define log_err(M, ...) W
    fprintf(stderr, "[ERROR] (%s:%d: errno: %s) " M "Wn", W
        __FILE__, __LINE__, clean_errno(), ##__VA_ARGS__)
```

출력 예시

```
[ERROR] (main.c:30: errno: No such file or directory) File open failed
```

[5-2] log_warn

```
[WARN] (file.c:45: errno: None) Something suspicious
```

[5-3] log_info

```
[INFO] (server.c:12) Server started
```

[6] check 매크로

```
#define check(A, M, ...) W
    if(!(A)) { W
        log_err(M, ##__VA_ARGS__); W
        errno = 0; W
        goto error; W
    }
```

동작

1. 조건 A 검사
2. 거짓이면:
 - 에러 로그 출력
 - errno 초기화
 - error: 라벨로 이동

사용 예시

```
FILE *file = fopen("test.txt", "r");
check(file != NULL, "File failed to open");

printf("SuccessWn");
return 0;

error:
    printf("Error occurredWn");
    return -1;
```


[7] sentinel 매크로

```
#define sentinel(M, ...) W
{ W
    log_err(M, ##__VA_ARGS__); W
    errno = 0; W
    goto error; W
}
```

용도

- 절대 실행되면 안 되는 코드에 사용

예:

```
switch(type) {
    case 1: break;
    case 2: break;
    default:
        sentienl("Invalid type");
}
```

[8] check_mem

```
#define check_mem(A) W
    check((A), "Out of memory.")
```

사용 예시

```
char *buf = malloc(100);
check_mem(buf);
```

NULL 이면 자동으로 error 로 이동

[9] check_debug

```
#define check_debug(A, M, ...) W
    if(!(A)) { W
        debug(M, ##__VA_ARGS__); W
        errno = 0; W
        goto error; W
    }
```

- check 와 동일
- 에러 메시지를 log_err 대신 debug 로 출력
- NDBUG 가 정의되면 메시지는 제거되지만 검사와 goto 는 그대로 동작

따라서 정리하자면

db.g 는 C 언어에서 디버깅과 에러 처리를 간결하고 일관되게 수행하기 위해 정의된 매크로 집합이다.

반복되는 에러 처리 코드를 줄이고, 파일명과 줄번호를 자동으로 출력하여 문제 추적을 쉽게 하는 것이 목적이다.

매크로	한 줄 용도
debug	개발 중 디버그 출력 (릴리즈 시 제거)
log_err	에러 메시지 출력
log_warn	경고 메시지 출력
log_info	일반 정보 출력
check	조건 실패 시 error로 이동
check_mem	메모리 NULL 검사
sentinel	오면 안 되는 코드 처리
check_debug	디버그용 조건 검사

Exercise 20. 고급 디버깅 방법

나는 이미 내가 만든 멋진 debug 매크로에 대해 가르쳐줬고, 너도 그것들을 사용해왔다. 내가 코드를 디버깅할 때는 debug() 매크로를 거의 전적으로 사용해서 무슨 일이 벌어지고 있는지 분석하고 문제를 추적한다.

이번 장에서는 실행 중이지만 종료되지 않는 간단한 프로그램을 GDB 로 조사하는 기본 방법을 가르쳐 줄 것이다.

너는 GDB 를 사용해서 실행 중인 프로세스에 붙고(attach), 그것을 멈추고, 내부에서 무슨 일이 일어나고 있는지 확인하는 법을 배우게 된다.

그 다음에는 GDB 를 사용할 때 도움이 되는 작은 팁과 요령들을 알려줄 것이다.

이 장은 영상 중심의 연습이다.

영상에서 내가 고급 디버깅 기법을 직접 보여준다.

아래 설명은 그 영상을 보강하는 내용이므로, 먼저 영상을 보는 것이 좋다.

디버깅은 내가 실제로 하는 것을 보는 것이 훨씬 배우기 쉽다.

[20-1] 디버그 출력 vs GDB

나는 디버깅을 할 때 기본적으로 “과학적 방법” 스타일로 접근한다.

가능한 원인을 가설로 세운 다음, 그것이 결함의 원인인지 증명하거나 배제한다.

많은 프로그래머들이 이 방식에 대해 가지는 문제는, 이것이 자신들을 느리게 만들 것 같다고 느낀다는 점이다.

그들은 버그를 빨리 해결하려고 당황하고 서두르지만, 그렇게 서두르는 동안 실제로는 아무런 유용한 정보를 수집하지 못한 채 허둥대고 있다는 사실을 깨닫지 못한다.

나는 로깅(디버그 출력)이 나로 하여금 과학적으로 버그를 해결하도록 강제한다고 느낀다. 또한 대부분의 상황에서 정보를 수집하기가 훨씬 쉽다.

내가 GDB 대신 디버그 출력을 주된 디버깅 도구로 사용하는 이유는 다음과 같다.

- 디버그 출력은 변수 값들을 포함한 프로그램 실행 전체의 추적(trace)을 보여준다. 이를 통해 무엇이 잘못되어 가는지 추적할 수 있다. GDB 에서는 원하는 모든 정보를 얻기 위해 여기저기에 watch 와 디버그 설정을 배치해야 하고, 실행 흐름 전체를 명확히 추적하기가 어렵다.

- 디버그 출력 코드는 소스에 그대로 남아 있을 수 있고, 필요할 때 다시 컴파일하면 바로 사용할 수 있다. GDB 에서는 버그마다 동일한 정보를 매번 새로 설정해야 한다.

- 서버에서 문제가 발생했을 때 디버그 로깅을 켜고 로그를 살펴보는것이 훨씬 쉽다. 시스템 관리자들은 로그를 다룰 줄 알지만 GDB는 모른다.

- 출력하는 것은 그냥 더 쉽다. 디버거는 항상 복잡하고 이상한 인터페이스와 불일치를 가지고 있다. 하지만

```
debug("Yo, dis right? %d", my_stuff);
```

처럼 출력하는 것은 전혀 복잡하지 않다.

- 디버그 출력을 작성하는 과정은 실제로 코드를 분석하고 과학적 방법을 사용하도록 강제한다. 디버그 사용은 이렇게 생각할 수 있다.

“나는 코드가 여기서 잘못되었다고 가설을 세운다.”
그리고 실행하면 그 가설이 검증된다.

만약 거기가 문제가 아니라면 다른 부분으로 이동한다.
겉보기에는 시간이 더 걸리는 것처럼 보일 수 있지만, 실제로는 차등 진단 과정을 통해 가능한 원인을 하나씩 제거하기 때문에 더 빠르다.

- 디버그 출력은 단위 테스트와도 잘 맞는다. 작업 중에는 디버거를 켜고 컴파일하고, 단위 테스트가 터지면 로그를 확인하면 된다. GDB를 쓰려면 단위 테스트를 GDB 아래에서 다시 실행하고 내부를 추적해야 한다.

이 모든 이유로 나는 debug를 GDB보다 더 의존하지만, 여전히 GDB를 사용하는 상황도 있다. 문제 해결에 도움이 된다면 어떤 도구든 가져야 한다고 생각한다.

어떤 경우에는 망가진 프로그램에 직접 붙여서 내부를 살펴봐야 한다.
또는 서버가 크래시하고 있고, 코어 파일(core file)만으로 원인을 확인해야 할 수도 있다.

이런 경우에는 GDB가 정답이다.
항상 가능한 많은 도구를 가지고 있는 것이 좋다.

[20-2] 언제 무엇을 쓰는가

- 나는 모든 메모리 오류를 잡기 위해 Valgrind를 사용한다. Valgrind가 문제를 일으키거나 너무 느리게 만든다면 GDB를 사용한다.

- 논리 오류나 사용 오류는 debug 출력으로 진단하고 수정한다. Valgrind를 사용하기 시작하면, 이후 버그의 90% 정도는 이 방식으로 해결된다.

- 그 외의 이상하고 설명하기 어려운 문제나 긴급 상황에서는 GDB 를 사용한다. Valgrind 도 아무것도 못 잡고, 필요한 정보조차 출력할 수 없다면 GDB 를 꺼내서 내부를 살펴본다. 이 경우 GDB 의 목적은 오직 “정보 수집”이다. 상황을 이해하면 다시 단위 테스트를 작성하고, print 문으로 왜 문제가 발생하는지 찾는다.

[20-3] 디버깅 전략

이 과정은 어떤 디버깅 기법을 사용하든 적용할 수 있다.

여기서는 사람들이 가장 많이 건너뛰는 GDB 기준으로 설명하겠다.

- notes.txt 라는 작은 텍스트 파일을 만들어서 실험 노트처럼 사용하자.
- GDB 를 사용하기 전에, 고치려는 버그와 그 원인이 무엇일 수 있는지 적어라.
- 가능한 각 원인에 대해, 관련 파일과 함수들을 적어라. 모르면 모른다고 적어라.
- 이제 GDB 를 시작하고, 가장 가능성이 높은 원인에 브레이크를 건다.
- 프로그램을 실행하여 그 원인이 맞는지 확인하라. set 명령으로 값을 바꿔서 쉽게 수정할 수 있는지, 혹은 오류를 즉시 재현할 수 있는지 확인하는 것이 가장 좋다.
- 그게 원인이 아니라면 notes.txt 에 아니라는 이유를 기록하라. 다음 가능한 원인으로 이동하고, 계속 정보를 추가하라.

눈치챘겠지만, 이것은 기본적으로 과학적 방법이다. 가설을 세우고, 디버깅으로 증명하거나 반박한다. 이 과정은 이미 배제한 원인을 반복해서 조사하는 일을 막아준다.

디버깅 출력으로도 같은 일을 할 수 있다.

차이점은 가설을 notes.txt 가 아니라 소스 코드에 작성한다는 점뿐이다. 어떤 의미에서는 디버깅 출력이 버그를 과학적으로 다루도록 강제한다.

[20-4] 추가 과제(Extra Credit)

- 그래픽 디버거를 찾아서 raw GDB 와 비교하자. 로컬 프로그램을 디버깅할 때는 유용하지만, 서버에서는 쓸모가 없다.
- 운영체제에서 core dump 를 활성화하자. 프로그램이 크래시하면 core 파일이 생성된다. 이 파일은 프로그램의 사후 분석 자료로, 크래시 순간에 무슨일이 있었는지 확인할 수 있다.

Exercise 20-1. 연습하기

[20-1] macOS 환경

mac OS 기본 디버거는 LLDB 이다.

[1] 정상 실행 코드

```
#include <stdio.h>
#include <unistd.h>

int main() {
    int i = 0;

    while(1) {
        printf("i = %d\n", i);
        i++;
        sleep(1);
    }

    return 0;
}
```

[2] 컴파일

컴파일:

```
clang -g ex20.c -o ex20
```

[3] 실행

```
./ex20
```

출력:

```
i = 0
i = 1
i = 2
...
```

[4] 실행 중 프로세스 확인

다른 터미널에서:

```
ps aux | grep ex20
```

- ps: 현재 실행 중인 프로세스(Process Status) 확인
- aux:
 - a: 모든 사용자 프로세스
 - u: 사용자 정보 포함
 - x: 터미널에 속하지 않은 프로세스까지 포함
- | : 파이프(앞 명령어 결과를 뒤로 전달)
- grep ex20: 출력 결과 중 ex20 이 포함된 줄만 필터링

또는

```
pgrep -fl ex20
```

PID 확인

[5] 실행 중 프로세스 attach

```
lldb -p PID
```

attach 하면 자동으로 멈춤(SIGSTOP 상태)

[6] 현재 스택 확인

```
bt
```

예시:

```
* thread #1, queue = 'com.apple.main-thread', stop reason = signal SIGSTOP
* frame #0: 0x00000001937e82f4 libsystem_kernel.dylib`__semwait_signal + 8
  frame #1: 0x00000001936c0d6c libsystem_c.dylib`nanosleep + 220
  frame #2: 0x00000001936c9ec0 libsystem_c.dylib`sleep + 52
  frame #3: 0x00000001025bc4ac ex20`main(argc=1, argv=0x000000016d843378) at
ex20.c:11:9
  frame #4: 0x000000019345dd54 dyld`start + 7184
```

[7] 왜 frame 3 인가?

- i 는 main 함수의 지역 변수
- main 이 frame #3 에 있음
- 따라서:

```
(lldb) frame select 3
```

```
(lldb) print i
```

[8] 실행 재개 및 중단

(lldb) continue

실행 중 Ctrl + C 누르면 해당 시점에서 멈춤

그 후:

```
bt
frame select 3
print i
```

그 순간의 i 값 확인 가능

[9] 크래시 유도 코드(macOS)

```
#include <stdio.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
    int i = 0;

    while(1){
        printf("i = %d\n", i);

        if (i == 5) {
            int *p = NULL;
            *p = 100;    // 의도적 크래시
        }

        i++;
        sleep(1);
    }
    return 0;
}
```

[10] 다시 컴파일

```
clang -g ex20.c -o ex20
```


[11] LLDB 로 실행

```
lldb ./ex20
```

```
(lldb) run
```

i == 5 에서 크래시 발생

출력 예시:

```
EXC_BAD_ACCESS (code=1, address=0x0)
```

[12] 크래시 분석

```
(lldb) bt
```

결과:

```
frame #0: main(argc=1, argv=0x0000000016fdff0e0) at ex20-1.c:13:16
```

NULL 포인터 역참조 확인

[20-2] Linux 환경

Linux 기본 디버거 GDB 이다.

[1] 컴파일

```
gcc -g ex20.c -o ex20
```

[2] 실행

```
./ex20
```

[3] 실행 중 attach

```
gdb -p PID
```

[4] 스택 확인

```
(gdb) bt
```

[5] 프레임 이동

```
(gdb) frame 3
```

```
(gdb) print i
```

[6] Linux 에서 Core Dump 분석

[6-1] Core dump 활성화

```
ulimit -c unlimited
```

확인

```
ulimit -c
```

[6-2] 실행 후 크래시

```
./ex20
```

출력:

```
Segmentation fault (core dumped)
```

core 파일 생성됨:

```
core
```

[6-3] Core 파일 분석

```
gdb ./ex20 core
```

분석

```
(gdb) bt
```

결과

```
#0 main at ex20.c:XX
```

크래시 위치 정확히 확인 가능

[20-3] C 프로그램 크래시 원인 분석 및 LLDB 디버깅 실습

[1] 크래시 발생 코드

```
if (i == 5) {  
    int *p = NULL;  
    *p = 100;    // 의도적 크래시  
}
```

위 코드에서 p 는 NULL(0x0)이다.

이를 역참조하여 값을 쓰려고 시도하면 메모리 접근 오류가 발생한다.

올바른 코드는 메모리를 먼저 할당해야한다.

```
int *p = malloc(sizeof(int));  
if(p != NULL) *p = 100;
```

[2] LLDB 실행 결과 분석

프로그램 실행 후 i == 5 에서 다음과 같은 메시지가 출력되었다.

```
* thread #1, queue = 'com.apple.main-thread', stop reason = EXC_BAD_ACCESS (code=1,  
address=0x0)  
  * frame #0: 0x00000001000004b4 ex20-1`main(argc=1, argv=0x000000016fdff0e0) at  
ex20-1.c:13:16  
    frame #1: 0x000000019345dd54 dyld`start + 7184
```

```
stop reason = EXC_BAD_ACCESS (code=1, address=0x0)
```

그리고

```
frame #0 main at ex20-1.c:13:16
```

[3] 이해해야 할 핵심

[3-1] EXC_BAD_ACCESS 의미

macOS 에서

- EXC_BAD_ACCESS = 잘못된 메모리 접근
- Linux 의 SIGSEGV 와 동일한 의미

즉, 프로그램이 접근하면 안 되는 메모리 영역을 접근했다는 뜻

[3-2] address=0x0 의미

```
address=0x0
```

0x0 은 NULL 주소이다.

따라서 다음이 확정된다.

- NULL 포인터를 역참조했다.
- 잘못된 주소(0 번 주소)에 쓰기 시도했다.

즉, NULL 포인터 역참조로 인한 크래시이다.

[3-3] frame #0 의미

frame #0 main at ex20-1.c:13:16

frame #0 은 CPU 가 실제로 예외가 발생한 위치

따라서, 정확한 분석 문장은 다음과 같다.

ex20-1.c 의 13 번째 줄에서 NULL 포인터를 역참조하여 EXC_BAD_ACCESS 가 발생하였다.

[4] 운영체제 관점에서의 동작 원리

[4-1] 왜 0x0 은 접근하면 안되는가

운영체제는

- 0 번 주소 영역을 의도적으로 매핑하지 않는다.
- NULL 포인터 오류를 즉시 감지하기 위함이다.

즉, 0 번 주소는 항상 invalid 상태이다.

이로 인해 프로그래머의 실수를 즉시 탐지할 수 있다.

[4-2] CPU 레벨에서 실제로 일어나는 일

1. CPU 가 메모리 쓰기 명령 실행
2. 대상 주소 레지스터 값 = 0x0
3. MMU 가 페이지 테이블 확인
4. 해당 주소에 매핑
5. 예외 발생
6. 운영체제가 EXC_BAD_ACCESS 전달
7. 프로세스 중단